

Digital Design & ASIC/FPGA Engineering

200 Master Technical Interview Questions & Solutions

Target Roles: ASIC/FPGA Digital Design Engineer · RTL Verification Engineer · Synthesis & STA Engineer · DFT Engineer · Physical Design (PnR) Engineer.

Curriculum Coverage: Verilog HDL · TCL · SpyGlass Lint/CDC · Design Compiler · PrimeTime STA · Low Power (UPF) · Asynchronous FIFO · Formal Verification (LEC/SVA) · DFT/ATPG/JTAG · GLS · PnR · FPGA.

Difficulty Level	Percentage	Questions	Target Competency
Advanced ADVANCED 60%	60%	120	Senior / Lead ASIC Architecture & Signoff
Moderate MODERATE 30%	30%	60	Mid-Level Engineering & Tool Automation
Easy EASY 10%	10%	20	Fundamental Digital Logic Principles
Total	100%	200	Comprehensive Interview Preparation

Contents

1	Verilog HDL & Digital System Architecture	2
2	TCL Scripting for EDA Automation	9
3	SpyGlass RTL Linting & RTL Quality	13
4	Logic Synthesis & Design Compiler	18
5	Static Timing Analysis (STA) & PrimeTime	24
6	Low Power Design & UPF	32
7	Clock Domain Crossing (CDC) & Async FIFO	38
8	Formal Verification & SystemVerilog Assertions (SVA)	47
9	Design for Testability (DFT, ATPG & JTAG)	51
10	Gate-Level Simulation (GLS) & SDF	57
11	Physical Design & Place & Route (PnR)	61
12	FPGA Architecture & ASIC vs FPGA	66

1 Verilog HDL & Digital System Architecture

Q1: Verilog Blocking vs Non-Blocking Assignment

EASY 10%

In synthesizable Verilog RTL for synchronous sequential logic (flip-flops), which assignment operator should be used, and in which IEEE simulation event region is its LHS update scheduled?

- A. =, Active region
- B. <=, NBA (Non-Blocking Assignment) update region
- C. =, Postponed region
- D. <=, Observed region

Detailed Solution & Technical Explanation

Correct Answer: B

In sequential `always @(posedge clk)` blocks, non-blocking assignments (`<=`) evaluate the RHS expression in the Active region and schedule the LHS variable update into the **NBA update region** of the IEEE 1364/1800 event queue. This guarantees deterministic behavior and eliminates race conditions between communicating sequential registers.

Q2: Register File Address Calculation

EASY 10%

For a dual-port register file with parameterized depth D , what is the correct standard Verilog-2001 system function to calculate the exact address bus width?

- A. `input [log2(D)-1:0] addr;`
- B. `input [$clog2(D)-1:0] addr;`
- C. `input [ln(D)/ln(2):0] addr;`
- D. `input [$log(D):0] addr;`

Detailed Solution & Technical Explanation

Correct Answer: B

`$clog2` is a built-in standard system function supported in Verilog-2001 and SystemVerilog that computes the ceiling of $\log_2(x)$, giving the exact number of address bits required to uniquely address D memory locations.

Q3: 3-Process FSM Registered Outputs

MODERATE 30%

Why is a 3-process (or registered-output) Moore/Mealy FSM preferred in high-performance ASIC design over a 2-process combinational-output FSM?

- A. It uses fewer flip-flops and reduces dynamic power.
- B. It completely eliminates combinational output glitches and provides a full clock period of timing budget to downstream modules.
- C. It eliminates the need for state encoding.
- D. It reduces the total number of FSM states by half.

Detailed Solution & Technical Explanation

Correct Answer: B

When FSM outputs are decoded combinatorially from state registers, intermediate decoding hazards cause glitches that consume dynamic power and eat into the downstream setup timing budget. Registering the outputs ensures glitch-free signals and creates clean register-to-register paths.

Q4: Inferred Latches in Combinational Blocks

MODERATE 30%

Consider the following combinational block:

```
always @(*) begin
    case (mode)
        2'b00: y = a + b;
        2'b01: y = a - b;
        2'b10: y = a & b;
    endcase
end
```

What hardware structure will logic synthesis infer, and why?

- A. Pure 4-to-1 Multiplexer with unused input tied to ground.
- B. Combinational AND-OR tree with y defaulting to zero.
- C. A transparent D-Latch on y because branch 2'b11 is missing and y must retain its previous value.
- D. A D Flip-Flop clocked by the mode signal.

Detailed Solution & Technical Explanation

Correct Answer: C

In an `always @(*)` block, if a variable is not assigned a value in all possible execution branches (e.g. missing 2'b11 without a `default:` statement), the synthesis tool infers a level-sensitive latch to preserve its state when `mode == 2'b11`.

Q5: Arithmetic vs Logical Shift Operators

MODERATE 30%

What is the key functional difference between `>>` (arithmetic right shift) and `>` (logical right shift) in Verilog?

- A. `>>` operates only on unsigned variables, filling MSBs with 0.
- B. `>>` sign-extends the MSB if and only if the operand is explicitly declared as `signed`.
- C. `>>` rotates bits through a carry register.
- D. Both operators always perform identical sign extension.

Detailed Solution & Technical Explanation

Correct Answer: B

In Verilog-2001, `>>` will only perform sign extension (filling empty MSB positions with the sign bit) if the operand being shifted is explicitly declared with the `signed` keyword. If the operand is unsigned, `>>` behaves identically to `>` by zero-filling.

Q6: LFSR Maximal Repetition Period

MODERATE 30%

An n -bit Linear Feedback Shift Register (LFSR) uses a primitive polynomial for maximal-length pseudo-random sequence generation. What is its exact repetition period in clock cycles?

- A. 2^n
- B. $2^n - 1$
- C. 2^{n-1}
- D. $n^2 - 1$

Detailed Solution & Technical Explanation

Correct Answer: B

A standard XOR-based LFSR has an illegal state of all zeros (00...0), which produces a lockup condition. Therefore, its maximum cycle length is $2^n - 1$.

Q7: 50% Duty Cycle Odd Clock Divider

MODERATE 30%

To generate a clean 50% duty cycle clock divided by an **odd** integer N (e.g., $N = 3$ or $N = 5$) from a reference clock without an internal PLL, what technique is required?

- A. Use an edge-triggered counter on positive edges only and toggle at $N/2$.
- B. Generate two intermediate clocks: one using positive clock edges and another using negative clock edges, each active for $(N - 1)/2$ cycles, and OR (or AND) them together.
- C. Divide the clock by $2N$ and use a frequency multiplier.
- D. Use an asynchronous delay line with an inverter ring.

Detailed Solution & Technical Explanation**Correct Answer: B**

Because an odd number cannot be divided into integers on positive edges alone, standard 50% duty cycle odd dividers generate two phase-shifted pulses (one triggered on **posedge** `ref_clk` and the other on **negedge** `ref_clk` with pulse width $(N - 1)/2$) and combine them with an OR/AND gate.

Q8: Signed ALU Overflow Condition

MODERATE 30%

In a 16-bit signed ALU performing two's complement addition $S = A + B$, which Boolean equation correctly flags signed arithmetic overflow?

- A. $V = A[15] \oplus B[15]$
- B. $V = (A[15] == B[15]) \& \& (S[15] \neq A[15])$
- C. $V = C_{out} \oplus A[15]$
- D. $V = S[15] \& (A[15] | B[15])$

Detailed Solution & Technical Explanation**Correct Answer: B**

In two's complement addition, overflow occurs if and only if adding two numbers of the same sign produces a result with the opposite sign (i.e. positive + positive = negative, or negative + negative = positive).

Q9: Verilog Event Queue Process Scheduling Race

ADVANCED 60%

Examine the following two **always** blocks:

```
// Block 1
always @(posedge clk) begin
    q1 = d;
end
// Block 2
always @(posedge clk) begin
    q2 = q1;
end
```

What is the value of `q2` after a clock edge where `d` changes from 0 to 1 (assuming initially `q1=0`, `q2=0`), and why is this non-deterministic?

- A. `q2` is guaranteed to be 0 because both blocks trigger simultaneously.
- B. `q2` is guaranteed to be 1 because Block 1 evaluates first in lexical order.
- C. Non-deterministic (Race condition): If Block 1 executes first, `q2` captures 1 (new value); if Block 2 executes first, `q2` captures 0 (old value).
- D. The simulator reports a compile-time syntax error.

Detailed Solution & Technical Explanation**Correct Answer: C**

Because blocking assignments (=) immediately update the LHS in the Active event region, the result depends entirely on the simulator's internal process scheduling order. Non-blocking assignments (<=) must always be used for sequential logic to prevent this race condition.

Q10: Multi-Driven Net Simulation vs Synthesis**ADVANCED 60%**

Consider two separate procedural blocks in the same module assigning to the same variable:

```
always @(posedge clk1) data_out <= in1;
always @(posedge clk2) data_out <= in2;
```

How do the simulator and logic synthesis tool handle this construct?

- A. Simulator treats it as wired-OR; Synthesis infers a clean 2-to-1 MUX.
- B. Simulator resolves it according to the last executed block or drives X; Synthesis tool halts with a **Multi-Driven Net / Multi-Source Error** (ELAB-900 / W422).
- C. Both simulator and synthesis tool automatically insert a clock domain arbiter.
- D. Simulator issues a fatal error; Synthesis infers a dual-clock flip-flop.

Detailed Solution & Technical Explanation**Correct Answer: B**

Multiple procedural blocks driving the same variable create multi-driver conflicts. In simulation, the last executed block overwrites the variable. Logic synthesis cannot map multiple unsynchronized clock drivers to a single physical register without multiplexing logic, triggering a fatal elaboration error.

Q11: Generate Loops and Scope Hierarchy**ADVANCED 60%**

In SystemVerilog/Verilog-2001, when instantiating parameterized processing elements inside a `genvar` loop:

```
genvar i;
generate
  for (i = 0; i < 4; i = i + 1) begin : gen_pe
    PE u_pe (.clk(clk), .din(data[i]), .dout(out[i]));
  end
endgenerate
```

What is the full hierarchical path to the internal register `acc_reg` inside the 3rd processing element (index 2)?

- A. `top.u_pe[2].acc_reg`
- B. `top.gen_pe[2].u_pe.acc_reg`
- C. `top.gen_pe.u_pe[2].acc_reg`
- D. `top.u_pe.gen_pe(2).acc_reg`

Detailed Solution & Technical Explanation**Correct Answer: B**

Named `generate` blocks (`begin : gen_pe`) create a generate scope array indexed by the `genvar` loop index. The correct hierarchy is `<module>.<generate_block_name>[<index>].<instance_name>.<signal>`.

Q12: Hazards of `full_case` and `parallel_case` Pragmas**ADVANCED 60%**

Why are Synopsys synthesis directives `// synthesis full_case` and `// synthesis parallel_case`

considered hazardous in modern ASIC design?

- A. They increase the cell count and silicon area by over 50%.
- B. They disable all static timing analysis checks.
- C. They cause **Simulation-Synthesis Mismatches** because the RTL simulator ignores comments while the synthesis tool alters its logic optimization.
- D. They force the placement tool to use only high-Vt cells.

Detailed Solution & Technical Explanation

Correct Answer: C

Synthesis pragmas inside comments (`// synthesis ...`) are invisible to standard RTL simulators (which evaluate the `case` statement strictly with priority or incomplete coverage). The synthesis tool synthesizes parallel multiplexers or don't-care states, resulting in silicon that behaves differently from the verified RTL simulation.

Q13: UART Receiver 16x Oversampling Timing

ADVANCED 60%

In a UART receiver operating with a 16x oversampling clock (`baud_16x_clk`), why is the start bit sampled at the 8th tick after the falling edge of `RX` instead of the 1st tick?

- A. To allow the line to charge up its capacitance.
- B. To sample at the theoretical midpoint (center) of the start bit, maximizing tolerance against clock jitter and baud rate drift, and to filter false glitches.
- C. To calculate the parity bit in advance.
- D. Because UART hardware requires 8 bits of preamble.

Detailed Solution & Technical Explanation

Correct Answer: B

16x oversampling checks for a valid falling edge at tick 0, verifies that `RX` remains low at tick 8 (the center of the start bit to reject high-frequency noise spikes), and subsequently samples each data bit at intervals of 16 ticks (the center of each data bit).

Q14: Parameterized Register File Write-Collision Handling

ADVANCED 60%

Consider a parameterized register file with synchronous write and combinational read:

```
always @(posedge clk) begin
    if (wr_en) mem[wr_addr] <= wr_data;
end
assign rd_data = mem[rd_addr];
```

What happens when `wr_addr == rd_addr` and `wr_en == 1` simultaneously?

- A. `rd_data` immediately reflects the new `wr_data` combinationaly in the same cycle (Write-Through / Read-First).
- B. `rd_data` outputs X due to memory collision.
- C. `rd_data` outputs the old value stored at `rd_addr` for the current cycle.
- D. The register file enters a deadlock state.

Detailed Solution & Technical Explanation

Correct Answer: A

Because the read port is a continuous assignment (`assign rd_data = mem[rd_addr]`), as soon as `mem[wr_addr]` is updated or if bypassed, the combinational read reflects the memory contents. However, to guarantee exact write-forwarding without intra-cycle races, explicit forwarding logic `assign rd_data = (wr_en && wr_addr == rd_addr) ? wr_data : mem[rd_addr];` is used.

Q15: One-Hot vs Binary FSM Encoding Trade-Offs

ADVANCED 60%

When targeting high-frequency ASIC designs with a large number of states (e.g. 20 states), why is **One-Hot** state encoding often preferred over **Binary** encoding?

- A. One-hot encoding minimizes the number of state flip-flops required ($N = \lceil \log_2 S \rceil$).
- B. One-hot encoding simplifies next-state and output combinational decoding logic to simple shallow gates (1-2 logic levels), maximizing clock frequency at the expense of more flip-flops.
- C. One-hot encoding produces zero dynamic switching power.
- D. One-hot encoding eliminates the need for reset logic.

Detailed Solution & Technical Explanation**Correct Answer: B**

Binary encoding requires complex multi-level decoding trees ($O(\log_2 S)$ depth) to evaluate next-state logic. One-hot encoding dedicates 1 flip-flop per state, allowing each state transition to be decoded using a simple 1-level AND/OR gate, dramatically reducing critical path delay for timing closure.

Q16: Latch-Based Integrated Clock Gating (ICG) Architecture

ADVANCED 60%

In an Integrated Clock Gating (ICG) cell for an active-high clock, why must the enable signal pass through a **negative-edge triggered latch** before feeding the AND gate?

- A. To synchronize the enable signal to an asynchronous domain.
- B. To hold the enable signal stable during the high phase of CLK, preventing positive clock glitches, hazard pulses, and shortened clock pulses.
- C. To invert the clock polarity for negative-edge flip-flops.
- D. To reduce leakage current during scan testing.

Detailed Solution & Technical Explanation**Correct Answer: B**

A simple AND gate without a latch produces clock glitches if **enable** toggles while CLK is high. By placing a latch that captures **enable** when CLK is low (transparent when CLK=0, latched when CLK=1), any transition on **enable** while CLK=1 is blocked, guaranteeing a glitch-free gated clock output.

Q17: SystemVerilog unique case vs priority case

ADVANCED 60%

What is the key verification and synthesis difference between **unique case** and **priority case** in SystemVerilog?

- A. **unique case** checks for full coverage only; **priority case** checks for overlap only.
- B. **unique case** asserts that exactly one condition is true at any time (flags a run-time warning on overlap or no match); **priority case** enforces evaluation order and only flags a warning if no condition matches.
- C. **priority case** synthesizes to a parallel MUX; **unique case** synthesizes to a priority encoder.
- D. They are strictly synonyms for backward compatibility with Verilog-95.

Detailed Solution & Technical Explanation**Correct Answer: B**

SystemVerilog **unique** instructs both simulator and synthesis tools that case arms are mutually exclusive (parallel) and fully specified; it produces run-time assertion warnings if multiple branches match or none match. **priority** enforces priority order and only checks that at least one branch matches.

Q18: Glitch Generation in Asynchronous Ripple Counters

ADVANCED 60%

Why are asynchronous ripple counters (where the output of stage N clocks stage $N + 1$) strictly forbidden in high-reliability synchronous ASIC design?

- A. They require twice the silicon area of synchronous counters.
- B. Cumulative clock-to-Q propagation delays through the ripple chain create severe output skew and intermediate combinational decoding glitches, causing false triggers in downstream logic.
- C. They cannot count backwards.
- D. They are not supported by Verilog simulators.

Detailed Solution & Technical Explanation

Correct Answer: B

In ripple counters, each flip-flop is clocked by the previous flip-flop, causing accumulated skew ($N \times t_{cq}$). When decoding multi-bit outputs (e.g., detecting state 4 $3'b100$ from $3'b011$), temporary transition states like $3'b010$ and $3'b000$ appear briefly, generating massive glitches that corrupt synchronous clock tree analysis and DFT scan chains.

Q19: Gray Code Mathematical Conversion

ADVANCED 60%

Given a 4-bit binary value $B = [B_3, B_2, B_1, B_0]$, what is the exact synthesizable expression to compute its corresponding Gray code $G = [G_3, G_2, G_1, G_0]$?

- A. `assign G = B ^ (B » 1);`
- B. `assign G = B & (B « 1);`
- C. `assign G = B + 1'b1;`
- D. `assign G = ~B ^ (B » 1);`

Detailed Solution & Technical Explanation

Correct Answer: A

Gray code conversion operates by XORing the binary word with its right-shifted version: $G_i = B_i \oplus B_{i+1}$, with $G_{MSB} = B_{MSB}$. In Verilog: `G = B ^ (B » 1)`.

Q20: System Controller Master FSM Transaction Sequencing

ADVANCED 60%

In the diploma's Master System Controller FSM, when a UART frame arrives requesting an ALU operation with operands, what is the correct state transition sequence?

- A. IDLE -> WRITE_REG -> EXEC_ALU -> SEND_UART -> IDLE
- B. IDLE -> READ_CMD -> READ_OPERANDS -> START_ALU -> WAIT_ALU_VALID -> SEND_UART_TX -> IDLE
- C. IDLE -> ALU_BUSY -> IDLE
- D. IDLE -> RESET -> EXEC -> IDLE

Detailed Solution & Technical Explanation

Correct Answer: B

The system controller FSM decodes the framing command, receives multi-byte operands via UART RX into temporary/register storage, asserts `ALU_ENABLE`, waits for `ALU_OUT_VALID`, and subsequently pulses `UART_TX_START` to transmit the result packet back through the asynchronous FIFO/UART TX interface.

Q21: Synthesizable Verilog function vs task

ADVANCED 60%

Which of the following statements correctly distinguishes a synthesizable Verilog **function** from

a task?

- A. Functions can contain `#` delay statements; tasks cannot.
- B. Functions execute in zero simulation time, return a single value directly, and cannot contain timing controls; tasks can have output/inout arguments and consume simulation time.
- C. Functions can invoke tasks, but tasks cannot invoke functions.
- D. Functions can drive global clock signals; tasks cannot.

Detailed Solution & Technical Explanation

Correct Answer: B

In Verilog HDL, a `function` represents pure combinational logic: it executes in zero simulation time, cannot contain time-consuming statements (`#`, `@`, `wait`), must have at least one input, and returns a single value. A `task` can consume time and return multiple values via `output/inout` arguments.

Q22: Structural Race Condition in Clock Multiplexing

ADVANCED 60%

What is the fatal flaw in multiplexing two asynchronous clocks using a simple combinational 2-to-1 MUX (`assign clk_out = sel ? clk_fast : clk_slow;`)?

- A. The synthesis tool converts it to an inverter chain.
- B. Switching `sel` asynchronously produces narrow runt pulses, clock glitches, and severe setup/hold violations on all downstream flip-flops.
- C. The MUX consumes excessive leakage power.
- D. The clock frequency is halved.

Detailed Solution & Technical Explanation

Correct Answer: B

Switching between unsynchronized clocks with a simple MUX creates sliver/runt clock pulses if `sel` toggles while either clock is high. A **Glitch-Free Clock Switch (GFCS)** circuit with cross-coupled negative-edge synchronizers and feedback suppression is required.

2 TCL Scripting for EDA Automation

Q23: TCL String & Variable Substitution Rules

EASY 10%

In TCL scripting for EDA tools, what is the exact output of `set a 5; set b {$a + 2}; puts $b`?

- A. 7
- B. 5 + 2
- C. \$a + 2
- D. Error: invalid syntax

Detailed Solution & Technical Explanation

Correct Answer: C

Curly braces `{}` in TCL suppress all variable (`$`) and command (`[]`) substitutions, treating the enclosed text as a literal string. Double quotes `"` or `expr` are required for evaluation.

Q24: EDA Tool Collections vs Standard TCL Lists

MODERATE 30%

In Synopsys Design Compiler or PrimeTime, why will `foreach port [get_ports *] { puts $port }` fail or produce unexpected results?

- A. The `puts` command does not work inside loops.
- B. `get_ports` returns an opaque tool **Collection Pointer** (handle), not a standard TCL list; `foreach_in_collection` must be used.
- C. `get_ports` only accepts single port names.
- D. Wildcards `*` are forbidden in Synopsys tools.

Detailed Solution & Technical Explanation

Correct Answer: B

Synopsys tools use internal memory-efficient structures called **collections**. Standard TCL commands like `llength` and `foreach` cannot parse collection pointers directly. The script must use `foreach_in_collection` or convert the collection using `get_object_name [get_ports *]`.

Q25: Querying Violating Timing Paths in PrimeTime

MODERATE 30%

Which TCL command correctly queries all timing paths whose setup slack is less than 0.0 in PrimeTime?

- A. `get_cells -filter "is_sequential == true && slack < 0.0"`
- B. `get_timing_paths -slack_lesser_than 0.0 -max_paths 100`
- C. `filter_collection [get_pins */CLK] "slack < 0"`
- D. Both B and a properly filtered `get_timing_paths` are standard EDA methods.

Detailed Solution & Technical Explanation

Correct Answer: D

In PrimeTime, `get_timing_paths -slack_lesser_than 0.0` directly queries violating paths, while `get_cells -filter` queries cell properties. Both leverage EDA collection filtering.

Q26: Regular Expressions in TCL for Report Parsing

MODERATE 30%

To extract the worst negative slack (WNS) value from a report line `Slack: -0.45 ns (VIOLATED)` into a TCL variable `wns_val`, which command is correct?

- A. `regexp {Slack:\s+([-d\.]+)\s+ns} $line match wns_val`
- B. `set wns_val [string index $line 8 12]`
- C. `regsub {Slack} $line wns_val`
- D. `scan $line "Slack: %s" wns_val`

Detailed Solution & Technical Explanation

Correct Answer: A

The regex `Slack:\s+([-d\.]+)\s+ns` searches for the literal word "Slack:", skips whitespace, captures the negative floating-point number into submatch group 1 (`wns_val`), and confirms trailing "ns".

Q27: Pass-By-Reference in TCL using upvar

MODERATE 30%

What is the primary function of `upvar 1 var_name local_alias` inside a TCL procedure?

- A. It promotes a local variable to a global environment across all files.
- B. It binds a variable in the caller's stack frame to a local name, enabling true pass-by-reference.
- C. It increments the integer value of `var_name` by 1.
- D. It imports environment variables from the Linux shell.

Detailed Solution & Technical Explanation**Correct Answer: B**

TCL procedures pass arguments by value. `upvar 1 <caller_var> <local_var>` creates an alias link to the caller's scope (1 level up the call stack), allowing the procedure to modify arrays or large data structures in place without copying.

Q28: Automated SDC Constraint Generator Budgeting**ADVANCED 60%**

Consider the following automated SDC constraint generator script:

```
set CLK_PERIOD 10.0
create_clock -name SYS_CLK -period $CLK_PERIOD [get_ports clk]
set_input_delay [expr $CLK_PERIOD * 0.25] -clock SYS_CLK [get_ports -filter "
    direction == in && name != clk"]
set_output_delay [expr $CLK_PERIOD * 0.30] -clock SYS_CLK [get_ports -filter "
    direction == out"]
```

What timing budget is left inside the chip for internal combinational logic between input ports and output ports on a purely feedthrough path?

- A. 10.0 ns
- B. 5.5 ns
- C. 4.5 ns
- D. 7.5 ns

Detailed Solution & Technical Explanation**Correct Answer: C**

The total clock period is 10.0 ns. External input delay consumes $25\% = 2.5$ ns. External downstream output delay consumes $30\% = 3.0$ ns. The remaining allowable internal datapath budget is $10.0 - 2.5 - 3.0 = 4.5$ ns (assuming ideal clock edges).

Q29: Accumulating Pin Capacitances on an Internal Net**ADVANCED 60%**

In Synopsys Design Compiler, which TCL script snippet accurately calculates the total pin capacitance connected to an internal net `n_data`?

- A. `set cap [get_attribute [get_nets n_data] capacitance]`
- B. `set cap [get_attribute [get_pins -leaf -of_objects [get_nets n_data]] capacitance]`
- C. `set cap 0; foreach_in_collection pin [get_pins -of_objects [get_nets n_data] -filter "direction == in"] { set cap [expr $cap + [get_attribute $pin capacitance]] }`
- D. `set cap [sizeof_collection [get_nets n_data]]`

Detailed Solution & Technical Explanation**Correct Answer: C**

The total load on a net consists of the sum of the input capacitances of all driven load pins (plus wire load). Iterating through the input pins of the net and querying their `capacitance` attribute using `get_attribute` accurately accumulates the pin load.

Q30: Safe Handling of Unconstrained Endpoints in Signoff**ADVANCED 60%**

You are writing a signoff checking script in PrimeTime. Which command correctly verifies that **no unconstrained timing endpoints** exist in the design?

- A. `report_constraint -all_violators`

- B. `check_timing -include {unconstrained_endpoints no_clock}`
- C. `report_timing -delay_type max -path_type full_clock`
- D. `get_timing_paths -slack_lesser_than 0.0`

Detailed Solution & Technical Explanation

Correct Answer: B

`report_timing` and `report_constraint` only evaluate constrained paths. If a register has a missing clock or unconstrained input/output, it is silently ignored by slack reports. `check_timing` explicitly identifies missing clocks, unconstrained endpoints, and loops.

Q31: Traversing Hierarchical Leaf Cells in TCL

ADVANCED 60%

When building an automated linting or cell-replacement script, how do you traverse all leaf-level instantiated library cells inside a hierarchical sub-block `u_sys`?

- A. `get_cells u_sys/*`
- B. `get_cells -hierarchical -filter "is_hierarchical == false" u_sys/*`
- C. `get_objects -all u_sys`
- D. `get_pins -of_objects u_sys`

Detailed Solution & Technical Explanation

Correct Answer: B

The `-hierarchical` flag recurses down all child levels, and filtering by `"is_hierarchical == false"` excludes parent boundary blocks, returning only fundamental leaf standard cells (AND, Flip-Flops, MUXes).

Q32: Error Trapping in Batch Synthesis Scripts

ADVANCED 60%

In an automated continuous-integration (CI) synthesis script, why is `if {[catch {compile_ultra} err_msg]} { puts "Synthesis Failed: $err_msg"; exit 1 }` used?

- A. `catch` runs the command in a separate background Linux thread.
- B. `catch` intercepts fatal exceptions and errors from the EDA command, preventing abrupt script termination and allowing clean log capture, error formatting, and return codes.
- C. `catch` forces the tool to ignore timing violations.
- D. `catch` accelerates tool execution speed.

Detailed Solution & Technical Explanation

Correct Answer: B

In TCL, unhandled command failures abort script execution immediately. The `catch` command executes the block, traps any abnormal return or syntax/runtime error, stores the error text into `err_msg`, and returns a non-zero exit code for programmatic handling.

Q33: Storing Timing Metrics in TCL Dictionaries

ADVANCED 60%

In PrimeTime, to construct a custom nested dictionary of startpoint-to-endpoint slacks: `dict set timing_db $sp $ep $slack`. What is the primary benefit of storing timing data in TCL 8.5+ `dict` structures versus standard arrays?

- A. Dictionaries are pass-by-value and can be nested, passed directly to procedures, and serialized easily without global scope collisions.
- B. Dictionaries run on GPU memory.
- C. Dictionaries automatically close setup timing violations.
- D. Dictionaries convert netlists to Verilog automatically.

Detailed Solution & Technical Explanation**Correct Answer: A**

TCL `dict` values are pure first-class values. Unlike classical TCL `arrays` (which are pass-by-name structures that cannot be returned from procedures or nested easily), `dict` objects support multi-level hierarchical nesting and clean functional data manipulation.

Q34: Automated EDA Log Parsing Matrix**ADVANCED 60%**

In a regression synthesis script running multiple runs with varying clock periods, what is the best practice for capturing runtime performance metrics?

- A. Manually opening each GUI session.
- B. Using `cputime` and `mem` attributes via `get_attribute [current_design] ...` combined with TCL file I/O to output a consolidated `.csv` summary matrix.
- C. Rerunning the tool with `+define+DEBUG`.
- D. Relying on standard terminal printouts without file redirection.

Detailed Solution & Technical Explanation**Correct Answer: B**

Automated regression flows invoke batch runs, extract metrics (WNS, TNS, Area, Dynamic Power, Runtime, Peak Memory) into structured key-value maps, and dump consolidated `.csv` / `.json` tables for signoff tracking.

3 SpyGlass RTL Linting & RTL Quality

Q35: Objective of RTL Linting in Design Flow**EASY 10%**

At which stage of the digital ASIC design flow is SpyGlass RTL Linting performed, and what is its primary objective?

- A. After Place-and-Route, to detect clock routing skew.
- B. Directly on RTL source code before synthesis, to detect syntax, structural bugs, latches, and simulation-synthesis mismatches.
- C. During fabrication, to measure silicon defect rates.
- D. Inside the FPGA bitstream programmer.

Detailed Solution & Technical Explanation**Correct Answer: B**

RTL Linting is static source code analysis performed before synthesis. It enforces coding rules, eliminates inferred latches, detects unconnected nets/ports, and identifies simulation-vs-synthesis divergences early in the flow.

Q36: SpyGlass Rule InferLatch (W188)**MODERATE 30%**

When SpyGlass triggers warning/error `InferLatch` (or `W188`) on variable `next_state`, what is the root cause?

- A. The variable is declared as a `wire` instead of `reg`.
- B. The variable is assigned in a combinational block that lacks complete assignments across all conditional branches (missing `else` or incomplete `case`).
- C. The flip-flop has an asynchronous reset.
- D. The clock frequency exceeds technology limits.

Detailed Solution & Technical Explanation**Correct Answer: B**

InferLatch / W188 indicates that a combinational always block fails to specify an output value for every possible input branch, forcing the synthesis tool to infer memory storage (a latch).

Q37: SpyGlass Rules W415 vs W416

MODERATE 30%

What design violations do SpyGlass rules W415 and W416 flag?

- A. W415: Clock pin unconstrained; W416: Reset pin unconstrained.
- B. W415: Blocking assignment (=) used in a sequential block; W416: Non-blocking assignment (<=) used in a combinational block.
- C. W415: Latch inferred; W416: Combinational loop detected.
- D. W415: Unconnected input; W416: Floating output.

Detailed Solution & Technical Explanation**Correct Answer: B**

W415 warns against blocking assignments in sequential `always @(posedge clk)` blocks (which cause race conditions). W416 warns against non-blocking assignments in combinational `always @(*)` blocks (which cause simulation event degradation and delta-cycle overhead).

Q38: Incomplete Sensitivity List (W18)

MODERATE 30%

Consider `always @(a or b) begin y = a & b | c; end`. Why does SpyGlass flag this with rule W18?

- A. Signal `c` is missing from the sensitivity list, causing the RTL simulation output `y` to remain unchanged when `c` toggles alone, whereas synthesized gates will evaluate `c` continuously.
- B. Variables `a` and `b` must be separated by commas in Verilog-95.
- C. Bitwise OR `|` is not synthesizable.
- D. The sensitivity list has too many signals.

Detailed Solution & Technical Explanation**Correct Answer: A**

An incomplete sensitivity list causes a simulation-synthesis mismatch. The simulator only triggers when `a` or `b` changes, ignoring changes in `c`. Logic synthesis ignores sensitivity lists entirely and builds combinational gates that respond immediately to `c`.

Q39: Unloaded and Undriven Nets (W528 and W240)

MODERATE 30%

Match the SpyGlass rule to its violation: 1. W240 2. W528

- A. (1) Signal assigned but never read; (2) Input port declared but unused.
- B. (1) Input port/signal declared but not driven/connected; (2) Variable set/assigned but never read downstream.
- C. (1) Multi-driven net; (2) Combinational loop.
- D. (1) Latch inferred; (2) Setup timing failure.

Detailed Solution & Technical Explanation**Correct Answer: B**

W240 flags undriven/floating inputs (floating nets can cause high gate leakage or unknown X states). W528 flags dead code/signals that are driven but never consumed downstream, indicating dead logic or missing connectivity.

Q40: Combinational Feedback Loops (W120 / W121)

ADVANCED 60%

Why are combinational feedback loops flagged with high-severity **Errors** in SpyGlass?

```
assign a = b & c;
assign b = a ^ d;
```

- A. They increase power dissipation by a factor of $10\times$.
- B. They create astable ring oscillators or latching states where timing analysis cannot determine topological paths, causing tool hang-ups, glitch oscillations, and unpredictable circuit states.
- C. They cause physical routing DRC violations.
- D. They cannot be converted into standard cell netlists.

Detailed Solution & Technical Explanation

Correct Answer: B

Combinational loops lack a synchronizing clock edge. They cause continuous high-frequency oscillations (glitch power) or unpredictable bi-stable memory storage. Furthermore, Static Timing Analysis (STA) tools cannot trace start-to-end paths through closed loops without breaking them artificially.

Q41: Bit-Width Mismatches and Truncation (W164a / W164b)

ADVANCED 60%

Consider the following RTL assignment:

```
wire [3:0] a;
wire [7:0] b = 8'hA5;
assign a = b;
```

What does SpyGlass rule W164a report, and what is the exact hardware consequence?

- A. Loss of 4 MSBs (8'hA is silently dropped, a gets 4'h5), which may cause functional arithmetic truncation bugs if not intended.
- B. The lower 4 bits are dropped.
- C. Synthesis automatically widens a to 8 bits.
- D. Synthesis ties all bits of a to zero.

Detailed Solution & Technical Explanation

Correct Answer: A

W164a flags LHS bit-width smaller than RHS (truncation). The upper 4 bits b[7:4] are dropped. If unintentional, this causes datapath corruption or overflow bugs.

Q42: Multi-Driven Bus Contention (W422 / W123)

ADVANCED 60%

When multiple tri-state drivers drive a shared internal on-chip bus without an active-enable decoder, what hazard does SpyGlass rule W422 prevent?

- A. Latch inference on bus lines.
- B. **Bus Contention** (multiple drivers simultaneously driving 0 and 1, causing excessive short-circuit current and thermal damage) and **Floating Bus** (no driver active, causing floating gates to drift into high leakage).
- C. Setup timing violations on the clock port.
- D. False reset deassertion.

Detailed Solution & Technical Explanation

Correct Answer: B

Tri-state internal buses can suffer from contention (short-circuit path between V_{DD} and V_{SS}) or floating states. Modern ASIC guidelines ban internal tri-state buses in favor of explicit multiplexer trees (MUX).

Q43: SpyGlass Lint Waiver Methodology in Signoff**ADVANCED 60%**

When is it permissible to add a rule waiver to a SpyGlass .awl (Advanced Waiver Language) file during ASIC signoff?

- A. Whenever a rule generates more than 100 warnings to clean up the report.
- B. Only after an engineer reviews the specific hierarchy, file, and signal, documents the exact technical rationale (e.g. intentionally unused status bits from a standard IP), and gets formal Lead Signoff approval.
- C. Waivers are never allowed in ASIC design under any circumstances.
- D. Only for timing violations on clock paths.

Detailed Solution & Technical Explanation**Correct Answer: B**

Waivers must never be used blindly to hide violations. Every waiver must be scoped specifically (by rule name, file, and line/hierarchy) and accompanied by a documented technical justification explaining why the condition is safe.

Q44: STARC Guidelines for Reset Synchronizers**ADVANCED 60%**

According to STARC (Semiconductor Technology Academic Research Center) design guidelines checked by SpyGlass, how must an asynchronous reset signal be connected to flip-flops?

- A. Routed directly from the external chip pad to all registers.
- B. Passed through a Reset Synchronizer (Asynchronously Asserted, Synchronously Deasserted) before driving internal register reset pins.
- C. Connected through a low-pass analog filter.
- D. Converted to a synchronous data input using an XOR gate.

Detailed Solution & Technical Explanation**Correct Answer: B**

Direct external asynchronous reset deassertion can occur near a clock edge, causing recovery/removal violations and metastability across flip-flops. STARC guidelines require reset synchronizers so reset asserts instantly but releases synchronously with the clock.

Q45: Structuring SpyGlass Sequential Goals**ADVANCED 60%**

What is the recommended sequential goal execution order in a comprehensive SpyGlass verification flow?

- A. lint_rtl → cdc_verify → constraints_verify → power_verify
- B. cdc_verify → lint_rtl → gls
- C. synthesis → lint_rtl → pnr
- D. dft_test → lint_rtl

Detailed Solution & Technical Explanation**Correct Answer: A**

Linting must always run first to guarantee clean syntax, types, and structure. Once RTL is lint-clean, CDC analysis checks multi-clock domains, followed by constraint verification and power intent validation.

Q46: Non-Synthesizable Constructs in RTL Code

ADVANCED 60%

Which of the following Verilog code segments will be flagged as an un-synthesizable construct by SpyGlass Lint?

- A. `always @(posedge clk or negedge rst_n)`
- B. `initial begin clk = 0; forever #5 clk = ~clk; end`
- C. `assign out = (sel) ? in1 : in0;`
- D. `always @(*) begin case (state) 2'b00: next = 2'b01; default: next = 2'b00; endcase end`

Detailed Solution & Technical Explanation

Correct Answer: B

`initial` blocks with delays (`#5`) and `forever` loops are simulation-only constructs used in testbenches; physical synthesis engines have no standard cell hardware equivalent for free-running procedural delays.

Q47: Multi-Bit Bus Re-convergence Lint Checks

ADVANCED 60%

Why does SpyGlass flag an error when individual bits of a multi-bit data bus (e.g. `data_bus[7:0]`) are synchronized across clock domains using independent 2-flip-flop synchronizers?

- A. It consumes too many flip-flops.
- B. Due to variable routing delays and cycle uncertainties (0 or 1 cycle latency variation), the synchronized bits can arrive in different destination clock cycles, creating corrupt intermediate data words (Data Incoherency).
- C. It causes clock jitter on the transmitter.
- D. It violates two's complement arithmetic rules.

Detailed Solution & Technical Explanation

Correct Answer: B

Multi-bit signals synchronized independently suffer from bit skew. If one bit is captured in cycle N and another in cycle $N + 1$, the destination receives invalid garbage values. Multi-bit buses must use Gray coding, Async FIFOs, or MUX-data handshakes.

Q48: Unreachable FSM States Detection

ADVANCED 60%

How does SpyGlass detect unreachable FSM states, and why is this critical for silicon area and safety?

- A. By checking power dissipation in sleep mode.
- B. By extracting the FSM state transition graph and performing formal reachability analysis from the reset state; unreachable states represent redundant logic that wastes silicon area or indicates a missing transition condition.
- C. By applying random vectors during gate-level simulation.
- D. By measuring clock tree skew.

Detailed Solution & Technical Explanation

Correct Answer: B

SpyGlass FSM analysis tools build the complete directed state graph. If any state cannot be reached via any sequence of transitions from the reset state, it flags a violation. Unreachable states indicate logic bugs and waste flip-flops/gates.

4 Logic Synthesis & Design Compiler

Q49: Logic Synthesis Transformation Stages

EASY 10%

Logic synthesis with Synopsys Design Compiler transforms RTL code into a technology-dependent gate-level netlist. What is the correct sequence of internal synthesis steps?

- A. Placement → Routing → CTS
- B. Elaboration / Translation (GTECH) → Logic Optimization (Boolean structuring) → Technology Mapping (Target .lib cells)
- C. ATPG → Scan Insertion → DRC
- D. Floorplanning → Extraction → Signoff

Detailed Solution & Technical Explanation

Correct Answer: B

Synthesis first parses and translates RTL into generic technology-independent gates (GTECH), optimizes Boolean equations to minimize logic depth and area, and then maps the optimized equations to specific standard cells from the target semiconductor library (.lib).

Q50: SDC create_clock Command Syntax

EASY 10%

Which SDC command correctly defines a 100 MHz clock named CORE_CLK with a 50% duty cycle on input port clk_in?

- A. `create_clock -name CORE_CLK -period 10.0 [get_ports clk_in]`
- B. `create_clock -frequency 100 -duty 50 clk_in`
- C. `set_clock CORE_CLK 10.0 [get_ports clk_in]`
- D. `create_clock -period 100.0 -waveform {0 50} [get_pins clk_in]`

Detailed Solution & Technical Explanation

Correct Answer: A

The time unit in standard SDC is nanoseconds ($100\text{ MHz} \implies T = 10.0\text{ ns}$). The default waveform is `{0 Period/2}` (50% duty cycle), so `-period 10.0` defines the 100 MHz clock.

Q51: Wire Load Models (WLM) vs Topographical Mode

MODERATE 30%

Why is standard synthesis using traditional Wire Load Models (WLM) inaccurate at deep sub-micron (DSM) technology nodes below 65 nm?

- A. WLMs overestimate gate leakage power.
- B. Interconnect resistance and capacitance dominate total path delay, and statistical WLMs fail to estimate actual physical wire lengths accurately, causing massive correlation discrepancies with post-PnR timing.
- C. WLMs do not support sequential cells.
- D. WLMs only work with FPGA architectures.

Detailed Solution & Technical Explanation

Correct Answer: B

In DSM nodes (45 nm, 28 nm, 7 nm), wire delay contributes > 60% of total path delay. Statistical WLMs guess wire lengths based on fanout alone. Synopsys DC-Topographical (`compile_ultra -spg`) uses physical floorplan and placement engines during synthesis to accurately predict real wire parasitics.

Q52: set_driving_cell vs set_drive

MODERATE 30%

Why is `set_driving_cell -lib_cell INV_X4 [get_ports din]` preferred over `set_drive 0 [get_ports din]` in realistic SDC synthesis scripts?

- A. `set_drive 0` sets the driving resistance to infinity.
- B. `set_driving_cell` accurately models the non-linear voltage-dependent drive resistance, output transition time (slew), and input capacitance from the exact standard cell library model.
- C. `set_driving_cell` automatically inserts a physical buffer on the PCB.
- D. `set_drive` is not supported in Design Compiler.

Detailed Solution & Technical Explanation

Correct Answer: B

Real input ports are driven by external chip pads or standard cells. `set_driving_cell` extracts the exact Non-Linear Delay Model (NLDM) or CCS timing tables of the driving cell to compute accurate input transition slews into the chip's first-stage logic.

Q53: Virtual Clocks in SDC Timing Constraints

MODERATE 30%

What is a **Virtual Clock**, and when is it necessary in SDC constraints?

- A. A clock that has no physical source pin/port in the design, used solely as a timing reference for constraining input arrival times and output delay requirements.
- B. A simulated clock generated inside a testbench.
- C. A clock operating at zero frequency.
- D. A clock used only for scan chain shifting.

Detailed Solution & Technical Explanation

Correct Answer: A

A virtual clock is defined using `create_clock -name VCLK -period 10.0` without specifying any source pin/port. It models the external transmitting or receiving device's clock for accurate `set_input_delay` and `set_output_delay` constraints.

Q54: set_clock_uncertainty Components in Pre-Layout

MODERATE 30%

During pre-layout logic synthesis, what physical effects does `set_clock_uncertainty` model?

- A. Power grid voltage drop only.
- B. Clock jitter (PLL phase noise) + estimated pre-CTS clock skew + clock tree synthesis margins.
- C. Leakage current variations.
- D. Thermal dissipation across package pins.

Detailed Solution & Technical Explanation

Correct Answer: B

Before physical Clock Tree Synthesis (CTS), clock trees do not exist (clocks are ideal). `set_clock_uncertainty` acts as a timing margin representing both the physical clock jitter (source instability) and the estimated clock skew that will appear post-CTS.

Q55: Synthesis Optimization Goals Priority

MODERATE 30%

In what order does Synopsys Design Compiler prioritize optimization constraints during `compile_ultra`?

- A. Area → Power → Dynamic Timing → DRCs
- B. Design Rule Constraints (DRCs: Max Capacitance, Max Transition, Max Fanout) → Setup Timing (Delay) → Area → Dynamic/Leakage Power
- C. Dynamic Power → Area → Setup Timing → DRCs
- D. Minimum Delay (Hold) → Maximum Delay (Setup) → Area

Detailed Solution & Technical Explanation

Correct Answer: B

Design Compiler strictly prioritizes electrical integrity: DRCs (Max Cap, Max Transition) must be satisfied first. Next is timing closure (WNS/TNS). Only after timing is met will the tool optimize for silicon area and power dissipation.

Q56: Operating Conditions & Slow PVT Corner Definition

MODERATE 30%

What does a "Worst-Case" (Slow) PVT corner represent for setup timing analysis in standard CMOS?

- A. High Voltage, Low Temperature, Fast Process
- B. Low Voltage (e.g. $0.9 \times V_{nom}$), High Temperature (e.g. $+125^{\circ}\text{C}$ in standard CMOS), Slow Process Corner (SS)
- C. Typical Voltage, 25°C , Typical Process (TT)
- D. Zero Voltage, 0°C , Fast Process (FF)

Detailed Solution & Technical Explanation

Correct Answer: B

Transistor switching speed degrades at low supply voltage (V_{DD}), slow process silicon (SS), and high junction temperature (125°C in traditional CMOS due to reduced carrier mobility), representing the maximum path delay for setup analysis.

Q57: Generated Clock Inversion and Division

ADVANCED 60%

A register `u_div2/q_reg` divides a 10 ns master clock `CLK` by 2 with an inverted output. Which SDC command correctly defines this generated clock?

- A. `create_clock -period 20.0 [get_pins u_div2/q_reg/Q]`
- B. `create_generated_clock -name GEN_CLK -source [get_ports clk] -divide_by 2 -invert [get_pins u_div2/q_reg/Q]`
- C. `set_clock_latency -source 2.0 [get_pins u_div2/q_reg/Q]`
- D. `create_generated_clock -period 20.0 [get_pins u_div2/q_reg/Q]`

Detailed Solution & Technical Explanation

Correct Answer: B

`create_generated_clock` maintains true phase and latency relationships with the master source clock. Specifying `-source [get_ports clk]`, `-divide_by 2`, and `-invert` accurately preserves clock tree path tracing and duty-cycle/polarity relationships in STA.

Q58: Clock Gating Insertion Transformation

ADVANCED 60%

When `compile_ultra -gate_clock` is enabled in Design Compiler, what transformation is performed on wide registers?

- A. Multiplexer feedback loops on register D-inputs are converted into latch-based Integrated Clock Gating (ICG) cells on the clock pin, saving dynamic power.

- B. Registers are replaced with dynamic domino logic.
- C. Clock buffers are removed to save area.
- D. Registers are converted into dual-edge triggered flip-flops.

Detailed Solution & Technical Explanation

Correct Answer: A

Standard RTL synchronous enable logic `if (en) q <= d;` synthesizes to a feedback MUX that keeps the clock toggling every cycle. Clock gating replaces the MUXes with an ICG cell that turns off the clock to the register bank whenever `en=0`, eliminating clock tree switching power.

Q59: Sequential Retiming Purpose

ADVANCED 60%

What is the effect of sequential **Retiming** during synthesis?

- A. It removes all flip-flops from the netlist.
- B. It automatically pushes/pulls flip-flops across combinational logic gates without altering the design's input/output transfer latency, balancing slack across stages.
- C. It adjusts the clock frequency during runtime.
- D. It replaces asynchronous resets with synchronous resets.

Detailed Solution & Technical Explanation

Correct Answer: B

Retiming relocates registers across logic gates to balance combinational delays between pipeline stages. This reduces the worst-case critical path delay (T_{clk}) and increases maximum operating frequency without changing the overall pipeline latency.

Q60: set_max_fanout vs set_max_capacitance DRCs

ADVANCED 60%

Why is `set_max_capacitance` a true physical DRC constraint while `set_max_fanout` is considered a design rule guideline?

- A. Capacitance limits are determined by transistor physical limits from the standard cell library characterization (`.lib`), directly affecting slew and reliability; fanout is a numerical gate count that does not account for variable wire lengths.
- B. Fanout is only used in FPGA synthesis.
- C. `set_max_capacitance` applies only to clock pins.
- D. Design Compiler ignores `set_max_capacitance`.

Detailed Solution & Technical Explanation

Correct Answer: A

Pin transition time and signal integrity depend on the physical capacitive load (farads). Fanout simply counts the number of load pins, ignoring wire capacitance and pin input capacitance differences. Standard cell `.lib` rules mandate `max_capacitance` limits to prevent cell damage and slew degradation.

Q61: False Paths vs Multicycle Paths Decisions

ADVANCED 60%

When should a path be constrained as a **Multicycle Path** rather than a **False Path**?

- A. When the path is physically impossible to trigger under any condition.
- B. When data is transferred across asynchronous clock domains without a handshake.
- C. When data requires multiple clock cycles (e.g. 2 or 3 cycles) to propagate through complex arithmetic logic, but is deliberately sampled only on cycle N via an enable signal.
- D. When hold time violation cannot be fixed.

Detailed Solution & Technical Explanation**Correct Answer: C**

A multicycle path is a functionally valid, synchronous path where the destination flip-flop is designed to capture data after $N > 1$ clock cycles. A false path tells the STA tool to completely ignore the path for all timing checks. Constraining a multicycle path as a false path prevents hold and setup verification.

Q62: Enforcing No-Latch Synthesis Rules**ADVANCED 60%**

How does Design Compiler optimize an RTL block when the user sets `hdlin_check_no_latch = true`?

- A. It replaces all latches with 2 flip-flops.
- B. It issues an immediate fatal compilation error if any latch is inferred during HDL translation, halting synthesis before mapping.
- C. It ties latch enable pins to V_{DD} .
- D. It converts latches into tri-state buffers.

Detailed Solution & Technical Explanation**Correct Answer: B**

`hdlin_check_no_latch` is a strict synthesis directive that causes the parser to halt with a fatal error if any combinational block infers a latch, enforcing clean RTL standards before technology mapping.

Q63: Critical Path Area Recovery Mechanism**ADVANCED 60%**

During the final phase of `compile_ultra`, how does Design Compiler perform **Area Recovery** on non-critical paths?

- A. By deleting redundant flip-flops.
- B. By down-sizing oversized high-drive standard cells (e.g. replacing `INV_X16` with `INV_X2` or `INV_X1`) on paths with positive slack, reducing area and dynamic power without violating setup timing.
- C. By reducing the supply voltage.
- D. By disabling scan chains.

Detailed Solution & Technical Explanation**Correct Answer: B**

To close timing on the critical path, synthesis uses large, high-drive, low- V_t cells. On non-critical paths with ample positive timing slack, the area recovery engine down-sizes cells to smaller sizes (and higher V_t) to minimize silicon area and leakage.

Q64: Boundary Optimization and Port Freezing**ADVANCED 60%**

What does the `set_boundary_optimization false [get_designs u_block]` command prevent Design Compiler from doing?

- A. It prevents the tool from synthesizing the internal logic of `u_block`.
- B. It preserves module port boundaries, preventing constant propagation across ports, hierarchical pin inversion, and merging of unused input/output ports.
- C. It disables place and route boundaries.
- D. It forces all ports to be registered.

Detailed Solution & Technical Explanation**Correct Answer: B**

Boundary optimization allows the synthesis tool to optimize across hierarchical sub-module boundaries (e.g., inverting complementary ports, propagating fixed constants into sub-blocks). Disabling it freezes the module interface, which is essential for Formal Verification (LEC) and hierarchical re-use.

Q65: Multi-Corner Multi-Mode (MCMM) Synthesis Architecture

ADVANCED 60%

Why is Multi-Corner Multi-Mode (MCMM) optimization essential in modern digital synthesis?

- A. To design chips that operate without clock signals.
- B. To simultaneously optimize the design across multiple operational functional modes (e.g. High-Performance Mode, Low-Power Sleep Mode, Scan Test Mode) and multiple PVT corners (Slow-Corner setup, Fast-Corner hold) in a single run.
- C. To combine FPGA and ASIC libraries together.
- D. To eliminate thermal dissipation.

Detailed Solution & Technical Explanation

Correct Answer: B

Designs have different operating modes (Functional, Scan Shift, Scan Capture, BIST) with conflicting timing paths, operating at different voltages and temperatures. MCMM synthesizes and closes timing across all modes and PVT corners concurrently, preventing fix-induced regressions.

Q66: set_dont_touch Constraints on Debug Cells

ADVANCED 60%

Why does an engineer apply `set_dont_touch [get_cells u_debug_reg]` during synthesis?

- A. To instruct Design Compiler not to optimize, remove, gate, or resize the specified cell or net, preserving it for silicon probe points or ECOs.
- B. To prevent the cell from being placed in the floorplan.
- C. To disable timing checks on that cell.
- D. To make the cell immune to radiation.

Detailed Solution & Technical Explanation

Correct Answer: A

Synthesis tools aggressively prune unused registers or constant logic. Setting `set_dont_touch` preserves critical debug registers, test structures, or precision analog-interface logic from being optimized away or modified.

Q67: Total Negative Slack (TNS) vs Worst Negative Slack (WNS)

ADVANCED 60%

In a synthesis timing summary report: $WNS = -0.15\text{ ns}$ and $TNS = -124.5\text{ ns}$. What do these metrics indicate to the ASIC designer?

- A. The design has only 1 violating path.
- B. The worst failing path violates setup timing by 0.15 ns, and there are hundreds of paths violating timing across the chip whose accumulated negative slack sums to -124.5 ns .
- C. The clock frequency must be reduced by 124.5 ns.
- D. The design passes all timing checks.

Detailed Solution & Technical Explanation

Correct Answer: B

WNS (Worst Negative Slack) is the maximum single timing violation in the design. TNS (Total Negative Slack) is the sum of negative slacks across all violating endpoints. A small WNS with large TNS indicates widespread timing closure issues requiring architectural or global constraint adjustments.

Q68: High-Fanout Net Synthesis and Ideal Networks

ADVANCED 60%

Why are clock and reset nets marked as `set_ideal_network` during logic synthesis in Design Compiler?

- A. To make synthesis finish faster by ignoring power consumption.
- B. Because high-fanout trees (clocks, resets) are physically synthesized and balanced later during Clock Tree Synthesis (CTS) in Physical Design; inserting unconstrained buffer trees during synthesis would distort real placement and waste area.
- C. Because ideal nets have infinite voltage.
- D. Because resets do not require routing wires.

Detailed Solution & Technical Explanation

Correct Answer: B

During synthesis, physical placement locations of millions of standard cells are not finalized. If synthesis attempted to build buffer trees for clocks or global resets, it would create suboptimal, un-routable trees. They are treated as ideal (zero delay, infinite drive) and built accurately during CTS.

5 Static Timing Analysis (STA) & PrimeTime**Q69: Fundamental Setup Timing Equation**

EASY 10%

For a register-to-register path with clock period T_{clk} , launch clock delay T_{launch} , capture clock delay $T_{capture}$, register clock-to-Q delay T_{cq} , combinational logic delay T_{comb} , and setup time T_{setup} , what is the condition for **Setup Timing** to be met?

- A. $T_{cq} + T_{comb} + T_{setup} \leq T_{clk} + (T_{capture} - T_{launch})$
- B. $T_{cq} + T_{comb} \geq T_{hold}$
- C. $T_{clk} \leq T_{setup} + T_{hold}$
- D. $T_{comb} = T_{clk}$

Detailed Solution & Technical Explanation

Correct Answer: A

Data launched at cycle 0 must arrive and settle at the capture register before the next clock edge at cycle 1 (T_{clk}). The arrival time is $T_{launch} + T_{cq} + T_{comb}$. The required time is $T_{capture} + T_{clk} - T_{setup}$. Rearranging gives $T_{cq} + T_{comb} + T_{setup} \leq T_{clk} + T_{skew}$, where $T_{skew} = T_{capture} - T_{launch}$.

Q70: Fundamental Hold Timing Equation

EASY 10%

What is the condition for **Hold Timing** to be satisfied on a synchronous register-to-register path?

- A. $T_{launch} + T_{cq} + T_{comb(min)} \geq T_{capture} + T_{hold}$
- B. $T_{comb(max)} \leq T_{clk} - T_{setup}$
- C. $T_{clk} \geq T_{hold}$
- D. $T_{skew} = T_{clk}$

Detailed Solution & Technical Explanation

Correct Answer: A

Data launched at cycle 0 must not propagate so fast that it corrupts data being captured at cycle 0 at the capture flip-flop. Thus, minimum arrival time $T_{launch} + T_{cq(min)} + T_{comb(min)}$ must be greater than or equal to $T_{capture} + T_{hold}$. Notice that clock period T_{clk} does not appear in the hold equation.

Q71: Clock Skew Impact on Setup and Hold

MODERATE 30%

Positive Clock Skew ($T_{capture} > T_{launch}$, where the capture clock edge arrives later than the launch clock edge):

- A. Helps Setup time but hurts Hold time.
- B. Hurts Setup time but helps Hold time.
- C. Hurts both Setup and Hold time equally.
- D. Has zero impact on timing closure.

Detailed Solution & Technical Explanation

Correct Answer: A

Positive skew provides extra time for data to propagate through combinational logic, thereby **increasing setup slack**. However, it increases the capture clock arrival time for the same clock edge, requiring data to remain stable longer, which **reduces hold slack** and can cause hold violations.

Q72: Setup vs Hold PVT Operating Corners

MODERATE 30%

Which operating corners are used for Setup and Hold timing signoff?

- A. Setup: Fast-Process, High-Voltage, Low-Temp; Hold: Slow-Process, Low-Voltage, High-Temp
- B. Setup: Slow-Process, Low-Voltage, High-Temp (SS/LowV/HighT - Max Delay); Hold: Fast-Process, High-Voltage, Low-Temp (FF/HighV/LowT - Min Delay)
- C. Setup and Hold are always checked at Typical (TT) conditions.
- D. Setup is checked at 0 K; Hold is checked at 300 K.

Detailed Solution & Technical Explanation

Correct Answer: B

Setup checks worst-case maximum datapath delay against the shortest effective period (slow silicon, low voltage, high temp). Hold checks best-case minimum datapath delay against fast clock transitions (fast silicon, high voltage, low temp), where fast data easily overwrites the previous state.

Q73: Clock Jitter vs Clock Skew Distinction

MODERATE 30%

What is the fundamental difference between **Clock Skew** and **Clock Jitter** in STA?

- A. Skew is dynamic cycle-to-cycle variation; Jitter is static spatial delay difference.
- B. **Clock Skew** is the spatial difference in arrival times between two different physical clock pins; **Clock Jitter** is the temporal (cycle-to-cycle or phase) variation in the clock edge arrival time at a single clock pin over time.
- C. Skew can be eliminated with PLLs; Jitter cannot.
- D. Skew affects only hold time; Jitter affects only setup time.

Detailed Solution & Technical Explanation

Correct Answer: B

Skew is spatial (difference in clock distribution routing latencies across the chip). Jitter is temporal (phase noise and cycle-to-cycle variation from the PLL and oscillator). Both reduce setup timing margins, but only skew directly impacts hold checks.

Q74: Recovery and Removal Asynchronous Checks

MODERATE 30%

What are **Recovery** and **Removal** timing checks in PrimeTime?

- A. Checks for memory read and write cycles.
- B. **Recovery** is the minimum time an asynchronous control signal (reset/preset) must be deasserted *before* the active clock edge (equivalent to Setup); **Removal** is the minimum time it must remain asserted *after* the active clock edge (equivalent to Hold).
- C. Checks for power-down sequencing.
- D. Checks for scan chain shift frequency.

Detailed Solution & Technical Explanation

Correct Answer: B

When releasing an asynchronous reset, if deassertion occurs too close to the active clock edge, flip-flops enter a metastable state or some flip-flops exit reset while others remain in reset. Recovery prevents setup-like violations, and Removal prevents hold-like violations on the reset pin.

Q75: Data-to-Data Timing Checks

MODERATE 30%

When is a `set_data_check` constraint applied in PrimeTime?

- A. Between two flip-flop outputs on a scan chain.
- B. To define setup and hold requirements between two asynchronous or non-clocked data signals (e.g. write-enable setup to address lines in an asynchronous SRAM interface).
- C. To check data bus bit-width matching.
- D. To verify parity bits in UART.

Detailed Solution & Technical Explanation

Correct Answer: B

Non-clocked interfaces (like asynchronous memory or custom handshakes) require setup and hold stability between two data signals (e.g. `data` valid before `write_enable` strobe). `set_data_check` instructs STA to enforce timing checks between two pure data pins.

Q76: Integrated Clock Gating Setup and Hold Checks

MODERATE 30%

For an active-high clock gating cell (AND gate driven by CLK and EN):

- A. EN must change only while CLK is high.
- B. EN must satisfy setup and hold timing relative to the **falling edge** of CLK before entering the latch, preventing glitches while CLK is high.
- C. No timing check is required for clock gating cells.
- D. EN must be tied to V_{DD} .

Detailed Solution & Technical Explanation

Correct Answer: B

In an active-high integrated clock gating cell (ICG), the latch is transparent when `CLK=0` and captures `EN` on the falling edge of `CLK`. STA verifies that `EN` arrives before the falling clock edge (Clock Gating Setup) and does not change prematurely after it (Clock Gating Hold).

Q77: Quasi-Static False Path Declaration

MODERATE 30%

Which of the following paths should be declared as a **False Path** (`set_false_path`)?

- A. The critical datapath in a 64-bit floating point adder.
- B. Static configuration register bits written once during boot-up and never toggled during normal execution.
- C. The scan enable distribution tree.

D. Memory read data bus during functional mode.

Detailed Solution & Technical Explanation

Correct Answer: B

Quasi-static configuration signals (set during chip power-up and constant during normal operation) do not switch dynamically cycle-to-cycle. Constraining them as false paths prevents the STA tool from wasting runtime and inserting unnecessary buffers on paths that never transition in operation.

Q78: SDC Multicycle Path Setup and Hold Modifiers

MODERATE 30%

A designer specifies a 2-cycle multicycle path:

```
set_multicycle_path 2 -setup -from [get_cells FF1] -to [get_cells FF2]
```

Without an explicit `-hold` command, what hold check does PrimeTime perform by default?

- A. Hold check at cycle 0 (the launch edge).
- B. Hold check at cycle 1 (one cycle before the new setup capture edge at cycle 2), which is overly restrictive and incorrect for a single-source clock.
- C. Hold checks are completely disabled.
- D. Hold check at cycle 2.

Detailed Solution & Technical Explanation

Correct Answer: B

By default in SDC, setting `-setup N` shifts the hold check to $N - 1$ cycles after the launch edge. For an actual multi-cycle datapath running on the same clock, the data is launched at cycle 0 and must not overwrite cycle 0 data, requiring: `set_multicycle_path 1 -hold -from [get_cells FF1] -to [get_cells FF2]`.

Q79: On-Chip Variation (OCV) Setup Slack Calculation

ADVANCED 60%

In an OCV setup analysis with $T_{clk} = 10$ ns, $T_{launch_clk} = 2.0$ ns, $T_{capture_clk} = 2.0$ ns, $T_{data} = 8.5$ ns, $T_{setup} = 0.5$ ns. If a **10% late derate** (1.10) is applied to the launch path and a **10% early derate** (0.90) is applied to the capture path: What is the resulting setup slack?

- A. +0.5 ns
- B. -1.25 ns
- C. -0.4 ns
- D. -0.9 ns

Detailed Solution & Technical Explanation

Correct Answer: C

- Derated Launch Arrival Time = $(2.0 \times 1.10) + (8.5 \times 1.10) = 2.2 + 9.35 = 11.55$ ns.
- Derated Capture Required Time = $10.0 + (2.0 \times 0.90) - 0.5 = 10.0 + 1.8 - 0.5 = 11.15$ ns.
- Setup Slack = Required - Arrival = $11.15 - 11.55 = -0.40$ ns (Violated).

Q80: Common Path Pessimism Removal (CPPR / CRPR)

ADVANCED 60%

Why is Common Path Pessimism Removal (CPPR) essential during OCV Static Timing Analysis?

- A. To increase the clock frequency of the PLL.
- B. To remove artificial timing pessimism caused by applying conflicting derating factors (e.g.

early derate and late derate simultaneously) to the identical, shared physical clock tree buffer segments feeding both launch and capture flip-flops.

- C. To compensate for board-level trace mismatches.
- D. To eliminate crosstalk glitches.

Detailed Solution & Technical Explanation

Correct Answer: B

A single physical clock buffer in the common clock tree cannot be physically fast (early) and slow (late) at the exact same instant. CPPR calculates the delay difference between early and late derates on the shared clock path and adds it back as a credit to the slack calculation.

Q81: Advanced OCV (AOCV) and Parametric OCV (POCV)

ADVANCED 60%

Why are AOCV and POCV significantly superior to flat global OCV derating at advanced FinFET nodes (16 nm, 7 nm, 3 nm)?

- A. Flat OCV derates assume worst-case variations across all cells globally, causing severe over-design. AOCV scales derates based on physical path depth and bounding-box distance (statistical averaging), while POCV models individual gate delay variations as statistical Gaussian distributions ($\mu \pm 3\sigma$).
- B. Flat OCV does not support multi-voltage domains.
- C. AOCV and POCV eliminate the need for standard cell characterization.
- D. AOCV runs without a timing engine.

Detailed Solution & Technical Explanation

Correct Answer: A

As logic depth increases, random independent process variations average out (Law of Large Numbers). Flat OCV applies a pessimistic uniform penalty (e.g. 15%). AOCV calculates lower derates for deeper paths and shorter distances, and POCV uses statistical cell-level sigma (σ) variations, eliminating excessive design margins.

Q82: Crosstalk Delay and Miller Coupling Capacitance

ADVANCED 60%

When two parallel on-chip metal tracks (Aggressor and Victim) switch simultaneously: What happens to the victim line delay when both switch in **opposite directions** versus **same direction**?

- A. Opposite: Delay decreases; Same: Delay increases.
- B. **Opposite directions**: Effective coupling capacitance doubles ($2 \times C_c$ via Miller effect), significantly **increasing victim transition delay**; **Same direction**: Effective coupling capacitance drops towards zero, **accelerating signal propagation** (speed-up / hold hazard).
- C. No change in delay occurs in either case.
- D. Both signals enter high-impedance state.

Detailed Solution & Technical Explanation

Correct Answer: B

Crosstalk Miller effect: Opposite-direction switching causes a $\Delta V = 2V_{DD}$ transition across the coupling capacitor C_c , doubling dynamic charge transfer and slowing down the victim (causing setup violations). Same-direction switching eliminates the voltage delta, speeding up the victim (causing hold violations).

Q83: Resolving Hold Violations vs Setup Violations

ADVANCED 60%

Why must hold timing violations be fixed exclusively by adding delay buffers on the datapath rather than increasing the clock period?

- A. Hold violations depend on the speed of light.
- B. The hold timing inequality ($T_{launch} + T_{cq} + T_{comb} \geq T_{capture} + T_{hold}$) is completely independent of the clock period T_{clk} ; slowing down the clock does not fix hold violations, and uncorrected hold violations result in un-testable, permanently broken silicon.
- C. Setup violations can only be fixed with inverters.
- D. Hold violations only occur during fabrication testing.

Detailed Solution & Technical Explanation

Correct Answer: B

Setup violations can always be resolved after tape-out by running the chip at a lower clock frequency (increasing T_{clk}). Hold violations are frequency-independent: if data races ahead of the same clock edge, the circuit fails at any clock frequency (1 kHz or 1 GHz). Hold must be closed clean in silicon.

Q84: Transparent Latches and Time Borrowing

ADVANCED 60%

How does a two-phase non-overlapping clock scheme with transparent latches achieve **Time Borrowing**?

- A. By accelerating the PLL frequency.
- B. If combinational logic preceding Latch 1 completes late (after the opening clock edge), the data can still propagate through the transparent latch during its open window, borrowing unused time from the subsequent clock phase without causing a timing failure.
- C. By borrowing power from unused blocks.
- D. By converting negative slack to positive slack using software.

Detailed Solution & Technical Explanation

Correct Answer: B

Unlike edge-triggered flip-flops that sample data strictly at a single instant, level-sensitive latches remain open and transparent for the entire active clock phase. If the input data arrives during the transparent phase, it passes through immediately, utilizing remaining slack in that phase.

Q85: Source-Synchronous Interface Advantages

ADVANCED 60%

Why are high-speed DDR memory interfaces (e.g. LPDDR5, DDR4) designed as **Source-Synchronous** interfaces?

- A. They use a single global oscillator located on the motherboard.
- B. The transmitting chip sends both the Data bus and a matched Clock/Strobe signal along identical PCB trace lengths, eliminating board-level clock skew and allowing operating speeds beyond several gigabits per second.
- C. They eliminate the need for read/write controllers.
- D. They operate with zero power.

Detailed Solution & Technical Explanation

Correct Answer: B

In common-clock (system-synchronous) interfaces, board-level clock skew between transmitter and receiver chips limits maximum frequency to ~ 200 MHz. In source-synchronous interfaces, the strobe (DQS) travels with the data (DQ), tracking matched delays and PVT drifts.

Q86: Path Grouping and Optimization Effort

ADVANCED 60%

Why does an STA engineer group timing paths into distinct path groups (`group_path -name CLK_GRP -critical_range 1.0`)?

- A. To randomize the order of timing checks.
- B. To prevent one dominant failing path group (e.g., a slow I/O interface) from starving other internal core register-to-register paths of optimization effort during synthesis and physical design.
- C. To disable timing checks on non-critical paths.
- D. To group registers by physical voltage domain.

Detailed Solution & Technical Explanation

Correct Answer: B

Optimization engines focus primarily on the worst overall negative slack. If I/O paths have -2.0 ns WNS, the tool will ignore internal paths with -0.2 ns WNS. Grouping paths ensures each category receives dedicated optimization effort.

Q87: Max Transition (Slew) Violations Severity

ADVANCED 60%

Why does PrimeTime report high severity for **Max Transition (Slew)** violations, even when setup and hold slacks are positive?

- A. Slow transitions cause large cell propagation delays, excessive dynamic short-circuit power (I_{sc} direct path from V_{DD} to V_{SS} during switching), and extreme susceptibility to crosstalk noise and EM failure.
- B. Max transition violations cause immediate bitstream corruption.
- C. Slew rates only affect scan chain testing.
- D. Max transition violations force the chip into reset mode.

Detailed Solution & Technical Explanation

Correct Answer: A

Slew violations mean signal edges rise/fall too slowly. During slow transitions, both NMOS and PMOS networks remain partially ON simultaneously, drawing massive short-circuit currents. Additionally, slow edges are easily perturbed by adjacent aggressor noise.

Q88: Pure Point-to-Point Min/Max Delay Constraints

ADVANCED 60%

When are `set_max_delay` and `set_min_delay` constraints used instead of `create_clock` and `set_input_delay`?

- A. For constraining pure asynchronous feedthroughs, custom analog IP interfaces, or point-to-point asynchronous crossing paths without reference clock periods.
- B. To define the master PLL frequency.
- C. To replace all setup and hold checks in a standard design.
- D. To fix scan chain hold violations automatically.

Detailed Solution & Technical Explanation

Correct Answer: A

`set_max_delay` and `set_min_delay` enforce absolute maximum and minimum path delays between start-points and endpoints without reference to clock edges or setup/hold relationships.

Q89: Half-Cycle Path Available Setup Budget

ADVANCED 60%

A path launches on the **rising edge** of CLK and is captured on the **falling edge** of the same CLK ($T_{clk} = 8.0\text{ ns}$, 50% duty cycle). What is the available clock budget for setup analysis (assuming zero skew and $T_{setup} = 0.4\text{ ns}$)?

- A. 8.0 ns
- B. 3.6 ns
- C. 4.0 ns
- D. 7.6 ns

Detailed Solution & Technical Explanation**Correct Answer: B**

In a rising-to-falling half-cycle path, the capture edge occurs at $T_{clk}/2 = 4.0\text{ ns}$. The available budget for $T_{cq} + T_{comb}$ is $4.0\text{ ns} - T_{setup} = 4.0 - 0.4 = 3.6\text{ ns}$.

Q90: Source Latency vs Network Latency in Clock Distribution

ADVANCED 60%

What is the distinction between **Source Latency** and **Network Latency** in PrimeTime?

- A. Source latency is on-chip clock tree delay; Network latency is PLL lock time.
- B. **Source Latency** is the delay from the primary clock generator/oscillator off-chip up to the chip's clock input port; **Network Latency** is the internal delay from the clock input port through the internal clock tree distribution network to the register clock pin.
- C. They are identical terms for wire resistance.
- D. Source latency applies only to reset signals.

Detailed Solution & Technical Explanation**Correct Answer: B**

Total clock latency = Source Latency (off-chip clock distribution) + Network Latency (internal on-chip clock tree insertion delay). Prior to CTS, both are set via `set_clock_latency`; after CTS, network latency is propagated from physical wires.

Q91: Transitioning to Propagated Clocks Post-CTS

ADVANCED 60%

When transitioning from pre-layout synthesis to post-CTS physical timing analysis in PrimeTime, which command must be issued to enable real clock tree delay calculations?

- A. `set_ideal_network -all`
- B. `set_propagated_clock [all_clocks]`
- C. `remove_clock_uncertainty -all`
- D. `set_clock_tree -enable`

Detailed Solution & Technical Explanation**Correct Answer: B**

In pre-CTS synthesis, clocks are ideal (zero insertion delay). Once the physical clock tree is synthesized in PnR, issuing `set_propagated_clock [all_clocks]` instructs the timing engine to compute actual network insertion delays, cell delays, and physical clock skew from real wire parasitics.

Q92: Temperature Inversion Phenomenon at Advanced Nodes

ADVANCED 60%

At advanced sub-40nm and FinFET process technologies, what is the **Temperature Inversion** phenomenon, and how does it impact STA signoff?

- A. Transistors run faster at high temperatures due to increased thermal energy.
- B. At lower supply voltages ($V_{DD} \approx V_{th}$), the reduction in threshold voltage (V_{th}) with decreasing temperature dominates over carrier mobility degradation, making standard cells **slower at cold temperatures (e.g. -40°C)** than at hot temperatures ($+125^{\circ}\text{C}$).
- C. Standard cells stop functioning at room temperature.
- D. Temperature inversion only affects analog PLL circuits.

Detailed Solution & Technical Explanation

Correct Answer: B

Traditionally, higher temperatures increased delay. In modern low-voltage nodes, mobility loss is outweighed by V_{th} shifts. Consequently, slow-corner setup analysis must be performed at both -40°C (Cold) and $+125^{\circ}\text{C}$ (Hot) to catch the true worst-case delay.

Q93: Silicon Failures from Reset Recovery Violations

ADVANCED 60%

What happens in silicon if a flip-flop violates its **Recovery Time** during asynchronous reset deassertion?

- A. The flip-flop burns out due to over-current.
- B. The flip-flop may enter a **metastable state** or experience a 1-cycle output resolution difference relative to other flip-flops, causing state machines to jump into invalid, corrupted states.
- C. The reset pin remains permanently locked to zero.
- D. The clock tree stops toggling.

Detailed Solution & Technical Explanation

Correct Answer: B

Recovery time is the setup requirement of an asynchronous pin relative to the clock. Violating recovery time causes internal feedback race conditions in the master/slave stages, producing non-deterministic metastability or staggered state updates across the chip.

Q94: Case Analysis Mode Configuration

ADVANCED 60%

What is the purpose of setting `set_case_analysis 0 [get_pins u_mux/sel]` in STA?

- A. It tests the circuit under all possible random conditions.
- B. It statically ties a pin/port to a constant logic state (0 or 1), disabling inactive functional paths through multiplexers and configuring the design for a specific operational mode (e.g. Functional Mode vs Scan Test Mode).
- C. It creates a high-impedance state on the bus.
- D. It checks for uninitialized variables.

Detailed Solution & Technical Explanation

Correct Answer: B

Designs share logic between functional mode, scan shift, scan capture, and BIST modes. `set_case_analysis` freezes mode-select pins to constant logic values, preventing STA from reporting unrealistic timing paths across inactive modes.

6 Low Power Design & UPF

Q95: CMOS Dynamic Power Formula

EASY 10%

What is the equation for CMOS dynamic switching power dissipation?

- A. $P_{dyn} = I_{leak} \times V_{DD}$
- B. $P_{dyn} = \alpha \cdot C_L \cdot V_{DD}^2 \cdot f$
- C. $P_{dyn} = V_{DD}/R$
- D. $P_{dyn} = \alpha \cdot C_L \cdot V_{DD} \cdot f^2$

Detailed Solution & Technical Explanation

Correct Answer: B

Dynamic power is consumed when capacitive loads charge and discharge: $P_{dyn} = \alpha \cdot C_L \cdot V_{DD}^2 \cdot f$, where α is switching activity factor, C_L is load capacitance, V_{DD} is supply voltage, and f is operating frequency.

Q96: Static Subthreshold and Tunneling Leakage

EASY 10%

What is the primary source of static (leakage) power in sub-micron CMOS circuits when no switching activity is occurring?

- A. Subthreshold leakage current (I_{sub}), gate oxide tunneling leakage (I_{gate}), and reverse-biased junction band-to-band tunneling (I_{junc}).
- B. Clock buffer switching capacitance.
- C. Resistor thermal Johnson noise.
- D. Package pin inductance.

Detailed Solution & Technical Explanation

Correct Answer: A

Static leakage occurs even when the clock is stopped. Subthreshold conduction ($V_{gs} < V_{th}$), gate-dielectric quantum tunneling, and junction leakage drain current continuously from V_{DD} to ground.

Q97: Multi-V_t Optimization Strategy

MODERATE 30%

How are Multi- V_t libraries (HVT, SVT, LVT) deployed across an ASIC design to minimize total power?

- A. LVT cells are placed everywhere to maximize speed.
- B. **LVT (Low- V_t)** cells (fastest switching, highest leakage) are used strictly on timing-critical paths to close setup time; **HVT (High- V_t)** cells (slowest switching, lowest leakage) are used on all non-critical paths with positive slack to suppress leakage power.
- C. HVT cells are used only for I/O pads.
- D. Multi- V_t requires three different power supplies.

Detailed Solution & Technical Explanation

Correct Answer: B

High- V_t cells have orders-of-magnitude lower leakage current than Low- V_t cells. Synthesis and PnR tools swap non-critical cells to HVT, maintaining target performance while drastically cutting standby leakage power.

Q98: Level Shifter Cell Insertion Requirements

MODERATE 30%

When is a **Level Shifter** cell required in a Multi-Voltage design?

- A. Between two flip-flops on the same voltage domain.
- B. At the boundary where a signal crosses from a **Low-Voltage Domain** (V_{DDL}) to a **High-Voltage Domain** (V_{DDH}) to prevent PMOS transistors in the receiver from staying

partially ON and drawing excessive static crowbar current.

- C. Only on clock signals.
- D. Inside the PLL circuit.

Detailed Solution & Technical Explanation

Correct Answer: B

When a 0.8 V signal drives a 1.2 V gate, a high output (0.8 V) cannot fully turn off the PMOS transistor (whose gate needs 1.2 V), creating a direct short-circuit path from V_{DDH} to ground. A Low-to-High level shifter shifts the voltage swing to full rail.

Q99: Power Gating Isolation Cells

MODERATE 30%

What is the role of an **Isolation Cell** at the interface of a Power-Gated domain?

- A. To step up the supply voltage.
- B. When a power domain is switched OFF (0 V), its floating output signals are clamped by isolation cells to a known, safe logic state (0 or 1) before reaching an active "Always-On" domain, preventing floating gates, invalid states, and crowbar currents.
- C. To isolate clock domains from data domains.
- D. To filter electro-magnetic interference.

Detailed Solution & Technical Explanation

Correct Answer: B

When power is cut to a power domain, its internal nodes drift into undefined intermediate floating voltages. Isolation cells clamp these boundary signals to constant safe values (0 or 1) so downstream active domains operate normally.

Q100: State Retention Power Gating (SRPG) Flip-Flops

MODERATE 30%

What is the purpose of a **Retention Flip-Flop (State Retention Power Gating - SRPG)**?

- A. To increase the maximum clock frequency.
- B. It contains a secondary "shadow" latch powered by an Always-On (V_{DD_AON}) supply that saves the register's state prior to main power shutdown and restores it immediately upon wake-up, bypassing lengthy system reboot/reconfiguration cycles.
- C. To retain data during scan testing.
- D. To prevent setup violations at low temperatures.

Detailed Solution & Technical Explanation

Correct Answer: B

Retention flip-flops preserve register states during sleep mode. Before power-gating V_{DD} , a **SAVE** pulse copies the state into an always-on shadow latch. Upon power-up, a **RESTORE** pulse copies the state back, enabling near-instantaneous wake-up.

Q101: Unified Power Format (UPF / IEEE 1801) Role

MODERATE 30%

What is the role of **UPF (IEEE 1801)** in the modern digital design flow?

- A. It is a programming language used to design analog PLLs.
- B. It provides a standardized TCL-based format to describe the low-power architectural intent (power domains, power switches, isolation strategies, level shifters, retention rules, power state tables) independently of the RTL HDL code.
- C. It replaces SDC timing constraints.

D. It is used exclusively for packaging thermal simulations.

Detailed Solution & Technical Explanation

Correct Answer: B

UPF allows architects to specify power gating, multi-voltage domains, isolation, and level shifting rules in a separate file. EDA tools (simulation, synthesis, formal verification, PnR) apply the UPF file to check and implement power management structures consistently.

Q102: Header (PMOS) vs Footer (NMOS) Power Switches

ADVANCED 60%

What is the trade-off between **Header Switches (PMOS)** and **Footer Switches (NMOS)** in power-gating architectures?

- A. Header switches use less area than footer switches.
- B. **NMOS Footer switches** (V_{SS} gating) have higher drive current and lower on-resistance (R_{on}) per unit area due to higher electron mobility, saving area; **PMOS Header switches** (V_{DD} gating) preserve substrate ground reference, simplifying noise management and multi-voltage interfaces.
- C. Header switches can only be used with HVT libraries.
- D. Footer switches do not support retention latches.

Detailed Solution & Technical Explanation

Correct Answer: B

NMOS electrons have $\approx 2 - 3\times$ higher mobility than PMOS holes, making NMOS footers physically smaller for the same I_{on} drop. However, footers bounce the virtual ground ($V_{SS_virtual}$), complicating substrate noise and level shifter design. PMOS headers keep true ground stable.

Q103: Inrush Current and Daisy-Chained Wakeup Sequences

ADVANCED 60%

When waking up a massive power-gated domain containing millions of gates: Why are the sleep transistors turned on in a staggered, **daisy-chained** sequence rather than simultaneously?

- A. To prevent excessive **Inrush Current** (di/dt) that causes severe power grid voltage drops (supply droop) on adjacent Always-On domains, which would otherwise induce false resets or timing failures.
- B. To allow clock trees to cool down thermally.
- C. Because the UPF standard limits power switches to 10 at a time.
- D. To give the PLL time to lock.

Detailed Solution & Technical Explanation

Correct Answer: A

Turning on all sleep transistors at once causes a massive current surge to charge depleted power-rail capacitances ($i = C \cdot dv/dt$). This inrush current creates an $L \cdot di/dt + I \cdot R$ voltage droop across the chip's power grid, resetting or corrupting active adjacent blocks. Daisy-chaining buffers turn on switches gradually.

Q104: Dynamic Voltage and Frequency Scaling (DVFS) Power Math

ADVANCED 60%

If a processor's supply voltage is reduced by 20% ($V_{new} = 0.8 V_{old}$) and frequency is reduced by 20% ($f_{new} = 0.8 f_{old}$), by what percentage is the dynamic power reduced?

- A. 20%
- B. 36%
- C. 48.8%

D. 64%

Detailed Solution & Technical Explanation**Correct Answer: C**

$P_{new} = \alpha C(0.8V)^2(0.8f) = 0.64 \times 0.8 \times P_{old} = 0.512 P_{old}$. Total Power reduction = $1 - 0.512 = 0.488 = 48.8\%$.

Q105: UPF Power State Table (PST) Verification

ADVANCED 60%

Consider the following UPF snippet:

```
create_pst sys_pst -supplies {VDD_CORE VDD_MEM VDD_AON}
add_pst_state S0 -pst sys_pst -state {FULL_ON FULL_ON FULL_ON}
add_pst_state S1 -pst sys_pst -state {OFF      FULL_ON FULL_ON}
add_pst_state S2 -pst sys_pst -state {OFF      OFF      FULL_ON}
```

What is the function of the Power State Table (`sys_pst`) during verification and synthesis?

- A. It defines the physical pad locations of power pins.
- B. It defines all valid combinations of power domain voltage states across operating modes, allowing automated tools to identify illegal voltage states and verify isolation/level-shifting rules at each boundary.
- C. It calculates the total static leakage in watts.
- D. It configures the CPU boot address.

Detailed Solution & Technical Explanation**Correct Answer: B**

The PST enumerates legal system power states. EDA tools use it to determine whether isolation cells, level shifters, or enable signals are required between domains for every reachable operational mode (e.g. Core Sleep, Memory Retention, Deep Standby).

Q106: Clock Gating Minimum Bit-Width Threshold Efficiency

ADVANCED 60%

Why do synthesis tools enforce a minimum bit-width threshold (e.g. `set_clock_gating_style -min_bitwidth 4`) before inserting an Integrated Clock Gating (ICG) cell?

- A. An ICG cell has an intrinsic cell area and internal clock pin capacitive load; for 1 or 2-bit registers, the power saved by gating the flip-flops is smaller than the overhead power consumed by the ICG itself.
- B. ICG cells cannot drive more than 1 flip-flop.
- C. It violates Verilog language semantics for 1-bit registers.
- D. Narrow registers do not have clock pins.

Detailed Solution & Technical Explanation**Correct Answer: A**

Inserting an ICG adds silicon area and continuous clock pin switching capacitance. For wide data buses (e.g. 16, 32, 64 bits), gating the bus saves massive power. For 1-2 bits, the overhead of the ICG exceeds the savings.

Q107: Glitch Power Reduction in Deep Arithmetic Pipelines

ADVANCED 60%

In deep combinational arithmetic structures (like carry-save multipliers or deep ripple adders), **glitch power** can account for up to 40% of total dynamic power. What design technique effectively eliminates glitch power?

- A. Removing all reset logic.
- B. Pipelining the datapath with balanced register stages to equalize path delays and stop intermediate transitions from propagating down the logic tree.
- C. Operating at higher supply voltages.
- D. Using larger driving cells.

Detailed Solution & Technical Explanation

Correct Answer: B

Glitches occur when inputs to a combinational gate arrive at different times due to unbalanced logic paths. Inserting balanced pipeline registers intercepts intermediate hazards, preventing glitches from propagating and charging downstream node capacitances.

Q108: Exponential Dependence of Subthreshold Leakage

ADVANCED 60%

Subthreshold leakage current is modeled by $I_{sub} \propto \exp\left(\frac{V_{GS}-V_{th}}{nV_T}\right)$. If the threshold voltage V_{th} of a standard cell is reduced by 100 mV at room temperature ($nV_T \approx 30$ mV): By approximately what factor does the subthreshold leakage current increase?

- A. $2\times$
- B. $5\times$
- C. $\approx e^{100/30} = e^{3.33} \approx 28\times$
- D. $100\times$

Detailed Solution & Technical Explanation

Correct Answer: C

Subthreshold leakage increases **exponentially** with decreases in V_{th} . A 100 mV reduction in V_{th} results in an increase of $e^{100/30} \approx 28\times$ in leakage power, illustrating the dramatic standby power penalty of Low- V_t (LVT) cells.

Q109: Isolation Control Signal Sourcing from Always-On Domain

ADVANCED 60%

When power-gating a domain PD_CPU, where must the `iso_en` (isolation enable) control signal be generated and powered from?

- A. From inside the PD_CPU domain using its switched power rail.
- B. From an **Always-On Power Domain** (V_{DD_AON}) so that the isolation control signal remains stable and asserted before, during, and after PD_CPU is powered down.
- C. From the external JTAG clock.
- D. From the memory bitlines.

Detailed Solution & Technical Explanation

Correct Answer: B

If the isolation enable signal originated from within the switched power domain, it would lose power and collapse when the domain is shut down, disabling isolation and allowing floating voltages to escape. Isolation controls must always originate from an Always-On domain.

Q110: Well Biasing (RBB vs FBB) Principles

ADVANCED 60%

What is the objective of **Reverse Body Biasing (RBB)** in low-power standby modes?

- A. To increase transistor switching speed during functional mode.
- B. Applying a reverse bias between the transistor body/well and source increases the effective threshold voltage (V_{th}) via the body effect, suppressing subthreshold leakage current by

up to $10\times$ during sleep.

- C. To reduce the gate oxide thickness.
- D. To eliminate wire resistance.

Detailed Solution & Technical Explanation

Correct Answer: B

Reverse Body Biasing (RBB) raises V_{th} when the chip is idle to cut leakage. Conversely, Forward Body Biasing (FBB) lowers V_{th} dynamically during high-performance bursts to maximize clock frequency at low operating voltages.

Q111: Always-On Buffers in Physical Power Routing

ADVANCED 60%

What is an **Always-On (AON) Buffer**, and when is it required during physical implementation?

- A. A buffer that switches continuously at 1 GHz.
- B. A special standard cell with a dedicated secondary V_{DD_AON} power pin, used when an always-on signal must be routed across a physically shut-down power domain floorplan tile.
- C. A buffer connected to an off-chip battery.
- D. A standard inverter used in clock trees.

Detailed Solution & Technical Explanation

Correct Answer: B

When routing an active signal across a physical floorplan area whose main power rail may be turned off, standard buffers placed in that tile would lose power. AON buffers connect to a continuous un-switched backup rail, enabling signal feedthrough across sleep regions.

Q112: Power-Aware Static Timing Analysis of Level Shifters

ADVANCED 60%

Why must Static Timing Analysis be power-aware when evaluating level-shifter paths?

- A. Level shifters have zero propagation delay.
- B. Level shifters introduce non-linear propagation delays that vary dramatically depending on the voltage delta between V_{DD_source} and V_{DD_dest} ; STA must evaluate the worst-case low voltage at source combined with high voltage at capture.
- C. Timing analysis is disabled in low-power domains.
- D. Level shifters eliminate setup checks.

Detailed Solution & Technical Explanation

Correct Answer: B

Level shifters operating with low input voltage take significantly longer to pull up/down the internal cross-coupled PMOS latches. STA tools must analyze paths under asymmetric multi-voltage corners ($V_{DDL_min} \rightarrow V_{DDH_max}$) to verify timing closure.

7 Clock Domain Crossing (CDC) & Async FIFO

Q113: Metastability Phenomenon in Digital Flip-Flops

EASY 10%

What physical phenomenon occurs when an asynchronous signal violates setup or hold time at a destination flip-flop?

- A. The flip-flop immediately catches fire.

- B. The flip-flop output enters a non-deterministic **Metastable State**, hovering between legal logic levels 0 and 1 for an unbounded duration before settling randomly.
- C. The flip-flop acts as a frequency divider.
- D. The signal voltage doubles.

Detailed Solution & Technical Explanation

Correct Answer: B

Violating setup or hold prevents internal feedback latches from resolving cleanly, leaving the internal nodes balanced at the threshold voltage. The output oscillates or hovers at an invalid intermediate voltage before thermal noise resolves it to 0 or 1.

Q114: 2-Flip-Flop Synchronizer Role

EASY 10%

What is the primary function of a standard **2-Flip-Flop (2-FF) Synchronizer**?

- A. To synchronize multi-bit parallel data buses.
- B. To provide a settling time of one full destination clock period ($T_{clk} - T_{setup}$) for a 1-bit asynchronous control signal to resolve metastability, reducing the probability of system failure.
- C. To double the clock frequency.
- D. To eliminate propagation delay completely.

Detailed Solution & Technical Explanation

Correct Answer: B

If the first flip-flop enters metastability due to an asynchronous transition, the second flip-flop samples its output one full clock cycle later, giving the first stage time to resolve to a stable binary logic level.

Q115: Mean Time Between Failures (MTBF) Scaling

MODERATE 30%

The MTBF for a synchronizer flip-flop is given by $MTBF = \frac{e^{t_{res}/\tau}}{T_0 \cdot f_{clk} \cdot f_{data}}$. To increase the MTBF from years to centuries:

- A. Increase the data frequency f_{data} .
- B. Increase the resolution settling time t_{res} (e.g. by using a 3-FF synchronizer or slower clock) and use flip-flops with small technology time constant τ .
- C. Decrease the clock period.
- D. Increase the operating temperature.

Detailed Solution & Technical Explanation

Correct Answer: B

Because t_{res} appears in the exponent ($e^{t_{res}/\tau}$), adding an extra synchronizer flip-flop (increasing resolution time by a full clock cycle) exponentially increases the MTBF, preventing system failures.

Q116: Fast-to-Slow Pulse Synchronizer Architecture

MODERATE 30%

Why will a 1-clock-cycle wide pulse in a Fast Clock Domain (1 GHz) be missed if sampled directly by a 2-FF synchronizer in a Slow Clock Domain (50 MHz)?

- A. The fast clock frequency causes electromagnetic interference.
- B. The pulse width (1 ns) is much shorter than the slow clock period (20 ns); if the pulse rises and falls between two consecutive slow clock edges, the synchronizer flip-flops never capture the pulse.

- C. 2-FF synchronizers only work from slow to fast domains.
- D. Slow domains reject all external pulses.

Detailed Solution & Technical Explanation

Correct Answer: B

For a 2-FF synchronizer to capture a signal, the input must remain stable for at least $1.5 \times T_{dest_clk}$ (Open-Loop rule). A fast pulse must be converted into a toggle signal (using a Toggle Synchronizer) or held by a handshake before being passed to the slow domain.

Q117: Multi-Bit CDC Synchronization Failure

MODERATE 30%

Why is passing a multi-bit binary counter across asynchronous clock domains using separate 2-FF synchronizers on each bit fatal?

- A. It consumes too much power.
- B. Due to wire routing skew and individual bit metastability resolution, when transitioning from 0111 (7) to 1000 (8), the destination domain may capture intermediate transient values like 1111 (15) or 0000 (0), corrupting system state.
- C. Binary counters cannot be synthesized.
- D. 2-FF synchronizers invert every odd bit.

Detailed Solution & Technical Explanation

Correct Answer: B

Multiple bits changing simultaneously will arrive at slightly different times at the destination registers due to wire delay skews. The destination synchronizer will sample a mixture of old and new bits, creating severe false data words.

Q118: Gray Code Single-Bit Transition Property in CDC

MODERATE 30%

Why is **Gray Code** universally used for FIFO pointer synchronization across asynchronous clock domains?

- A. Gray code uses fewer bits than binary.
- B. Exactly **one bit changes per increment/decrement transition**, ensuring that even with arbitrary clock edge alignment and routing skew, the synchronizer samples either the previous pointer or the new pointer, with zero possibility of corrupted intermediate values.
- C. Gray code operates without a clock signal.
- D. Gray code automatically clears FIFO full flags.

Detailed Solution & Technical Explanation

Correct Answer: B

Because only a single bit toggles between adjacent Gray code numbers ($00 \rightarrow 01 \rightarrow 11 \rightarrow 10$), there are no intermediate transition states. The synchronizer either captures the old pointer value or the new one—both of which are safe and valid.

Q119: Reset Synchronizer Asynchronous Assert / Synchronous Deassert

MODERATE 30%

Why does an industry-standard **Reset Synchronizer** assert reset asynchronously but deassert it synchronously?

- A. To save silicon area on the reset pad.
- B. Asynchronous assertion guarantees instant shutdown upon power-on or fault even if the clock is stopped; Synchronous deassertion guarantees that reset release satisfies flip-flop recovery/removal times relative to the active clock edge.

- C. To allow the CPU to run without a reset signal.
- D. To eliminate reset buffers.

Detailed Solution & Technical Explanation

Correct Answer: B

Immediate asynchronous assertion protects the circuit without waiting for a clock. Releasing the reset through 2 flip-flops clocked by the target clock ensures the deassert edge is aligned with the clock, preventing recovery timing violations.

Q120: CDC Re-convergence Hazards

MODERATE 30%

What is **CDC Re-convergence**, and why does SpyGlass CDC flag it as an error?

- A. When a clock tree divides into two branches.
- B. When two independently synchronized control signals originating from the same source domain are combined combinational-wise in the destination domain; cycle uncertainties (0 or 1 cycle delay in each synchronizer) can cause the signals to arrive out-of-sync, corrupting the decoded state.
- C. When a reset signal drives a clock pin.
- D. When two PLLs are connected in series.

Detailed Solution & Technical Explanation

Correct Answer: B

Even if two 1-bit signals are individually synchronized via 2-FF synchronizers, one may resolve in 1 cycle while the other resolves in 2 cycles. Combining them with logic in the destination domain produces invalid decoded states. They must be merged into a single state/handshake before crossing.

Q121: Asynchronous FIFO Full and Empty Domains

ADVANCED 60%

In a dual-clock Asynchronous FIFO with synchronized Gray pointers: In which clock domain are the **FIFO Empty** and **FIFO Full** flags evaluated?

- A. Empty in Write domain; Full in Read domain
- B. **Empty in Read Clock Domain** (comparing local read Gray pointer with synchronized write Gray pointer); **Full in Write Clock Domain** (comparing local write Gray pointer with synchronized read Gray pointer).
- C. Both flags are evaluated in a third asynchronous monitoring domain.
- D. Both flags are evaluated purely combinatorially without clocks.

Detailed Solution & Technical Explanation

Correct Answer: B

Empty controls reading, so it must be evaluated in the Read domain with zero latency to prevent reading an empty FIFO. **Full** controls writing, so it must be evaluated in the Write domain with zero latency to prevent overwriting a full FIFO.

Q122: Asynchronous FIFO Full Flag Boolean Condition

ADVANCED 60%

For an Async FIFO of depth 2^N using $(N+1)$ -bit Gray code pointers W_{ptr} and R_{ptr_sync} : What is the exact condition that indicates the FIFO is **Full**?

- A. $W_{ptr} == R_{ptr_sync}$
- B. $W_{ptr}[N : N-1] == \sim R_{ptr_sync}[N : N-1]$ and $W_{ptr}[N-2 : 0] == R_{ptr_sync}[N-2 : 0]$ (MSB and 2nd MSB inverted, all remaining LSBs identical).

- C. $W_{ptr} - R_{ptr_sync} == 2^N$
 D. $W_{ptr}[N] == R_{ptr_sync}[N]$

Detailed Solution & Technical Explanation

Correct Answer: B

Using an extra pointer bit ($(N+1)$ bits for depth 2^N), when pointers wrap around (write has lapped read by exactly 2^N), the MSB and 2nd MSB of the Gray code pointers are inverted ($\sim$), while the remaining lower $N-1$ bits match identically. (If all bits match, the FIFO is Empty).

Q123: Pessimistic Safe Behavior of Async FIFO Flags

ADVANCED 60%

Because pointer synchronization across clock domains introduces a 2-cycle latency: What is the operational safety implication on the **Full** and **Empty** flags in an Async FIFO?

- A. The FIFO may overflow or underflow by 2 words.
 B. The flags are **pessimistic (safe)**: **Full** may stay asserted for 2 extra write cycles after a read occurs (temporarily stalling writes when not strictly full), and **Empty** may stay asserted for 2 extra read cycles after a write occurs; but the FIFO will **never overflow or underflow**.
 C. The FIFO drops random data packets.
 D. The write pointer resets to zero automatically.

Detailed Solution & Technical Explanation

Correct Answer: B

The write domain sees a delayed version of the read pointer, so it believes the FIFO has fewer free spaces than it actually does (safe). Similarly, the read domain sees a delayed write pointer, believing fewer words are available (safe).

Q124: Asynchronous FIFO Burst Depth Calculation

ADVANCED 60%

Calculate the minimum Async FIFO depth for the following specifications:

- **Write Clock:** $f_w = 200 \text{ MHz}$ ($T_w = 5 \text{ ns}$)
- **Read Clock:** $f_r = 50 \text{ MHz}$ ($T_r = 20 \text{ ns}$)
- **Burst Length:** $B = 80$ contiguous data words
- **Read Duty:** 1 read operation every clock cycle (1 read/20 ns)
- **Write Duty:** 1 write operation every clock cycle (1 write/5 ns)

What is the minimum FIFO depth required to prevent data loss during the burst?

- A. 20 words
 B. 40 words
 C. 60 words
 D. 80 words

Detailed Solution & Technical Explanation

Correct Answer: C

1. Time to write burst = $B \times T_w = 80 \times 5 \text{ ns} = 400 \text{ ns}$.
2. Number of reads during burst = $\text{Time}/T_r = 400 \text{ ns}/20 \text{ ns} = 20$ words read out.
3. Minimum FIFO Depth = Burst – Reads = $80 - 20 = 60$ words. (Standard power-of-2 implementation uses Depth = 64).

Q125: Non-Power-of-2 FIFO Depth Gray Code Pitfall

ADVANCED 60%

Why is implementing an Asynchronous FIFO with a non-power-of-2 depth (e.g. Depth = 10) dangerous with standard Gray code pointers?

- A. Non-power-of-2 memories cannot be manufactured in silicon.
- B. Standard Gray code sequences only exhibit single-bit transitions when wrapping around from $2^N - 1$ back to 0. Wrapping around at an arbitrary number (e.g. from index 9 back to 0) causes **multiple bits to toggle simultaneously**, destroying the single-bit CDC property and causing metastability bugs.
- C. Synthesis tools reject non-power-of-2 arrays.
- D. It requires 5-stage synchronizers.

Detailed Solution & Technical Explanation

Correct Answer: B

Gray code symmetry is preserved only across 2^N counts. Wrapping from 9 (4'b1101) to 0 (4'b0000) changes 3 bits in a single step, re-introducing multi-bit CDC race conditions. Specialized even-count truncated Gray sequences are mandatory for non-power-of-2 depths.

Q126: 4-Phase vs 2-Phase CDC Handshake Protocols

ADVANCED 60%

What is the difference between **4-Phase (Level-based)** and **2-Phase (Transition-based)** CDC handshakes?

- A. 4-phase handshakes require 4 data wires; 2-phase handshakes require 2 data wires.
- B. **4-Phase Handshake:** REQ asserts → ACK asserts → REQ deasserts → ACK deasserts (returns to zero, taking longer latency); **2-Phase Handshake:** Any toggle on REQ triggers a transfer, and any toggle on ACK confirms it (faster throughput, higher logic complexity).
- C. 4-phase is only for synchronous domains.
- D. 2-phase handshakes do not require synchronizers.

Detailed Solution & Technical Explanation

Correct Answer: B

4-phase handshakes require a full return-to-zero cycle for both REQ and ACK (4 crossing transitions per transaction). 2-phase (Non-Return-to-Zero) handshakes treat each transition (0-to-1 or 1-to-0) as an active event, cutting transaction latency in half.

Q127: Fast-to-Slow Domain MUX-Data Handshake Synchronizer

ADVANCED 60%

In a MUX-Data CDC synchronizer (Data-Load / Handshake Enable): How is a 64-bit multi-bit datapath safely transferred from a fast to a slow domain without Gray coding?

- A. The data bus is routed directly to the destination registers, while an independent `data_valid` pulse is stretched, synchronized through a 2-FF synchronizer to the slow domain, and used as the MUX/Clock-Enable to safely sample the stable data bus.
- B. All 64 bits are passed through 64 independent 2-FF synchronizers.
- C. By reducing the fast clock frequency to zero.
- D. By converting the 64-bit data to serial UART data.

Detailed Solution & Technical Explanation

Correct Answer: A

The transmitter places data on the bus and asserts a control flag. The data bus remains completely static while the control flag is synchronized via 2 flip-flops. Once the synchronized enable arrives in the

destination domain, the destination register samples the stable bus with zero metastability risk.

Q128: SpyGlass CDC Rules Ac_unsync01 and Ac_conv01

ADVANCED 60%

Match the SpyGlass CDC violation rules: 1. Ac_unsync01 2. Ac_conv01

- A. (1) Unsynchronized crossing from source to destination domain; (2) Re-convergence of synchronized signals in destination domain.
- B. (1) Latch inferred; (2) Reset glitch.
- C. (1) Clock domain mismatch; (2) Scan chain bypass.
- D. (1) Multi-driven net; (2) Combinational loop.

Detailed Solution & Technical Explanation

Correct Answer: A

Ac_unsync01 flags direct structural CDC crossings lacking a valid synchronizer (2-FF, FIFO, handshake). Ac_conv01 flags multiple synchronized signals converging into combinational logic in the destination domain.

Q129: Reset Domain Crossing (RDC) Metastability

ADVANCED 60%

What is a **Reset Domain Crossing (RDC)** bug, and how does it occur even when all flip-flops operate on the exact same synchronous clock?

- A. When a flip-flop clock is disconnected.
- B. When Flip-Flop A is reset by rst_a_n while communicating with Flip-Flop B which is reset by rst_b_n; when rst_a_n asserts asynchronously, the sudden transition on Flip-Flop A's output can violate setup/hold times of Flip-Flop B, inducing metastability.
- C. When a reset signal has high fanout.
- D. When synchronous resets are used instead of asynchronous resets.

Detailed Solution & Technical Explanation

Correct Answer: B

Asynchronous reset assertions occur independently of the clock edge. If two registers share a clock but have independent reset domains, asserting reset on the transmitter acts as an asynchronous data transition into the receiver, causing RDC metastability.

Q130: SDC set_clock_groups -asynchronous Meaning

ADVANCED 60%

What is the effect of issuing the following SDC constraint in PrimeTime:

```
set_clock_groups -asynchronous -group [get_clocks CLK_A] -group [get_clocks CLK_B]
```

- A. It aligns the phase of CLK_A and CLK_B.
- B. It instructs the Static Timing Analysis engine to completely disable all setup and hold timing checks between all paths crossing between domain CLK_A and domain CLK_B.
- C. It inserts 2-FF synchronizers automatically during synthesis.
- D. It generates a clock divider.

Detailed Solution & Technical Explanation

Correct Answer: B

set_clock_groups -asynchronous informs the STA tool that no fixed phase relationship exists between CLK_A and CLK_B. STA cuts all inter-clock timing paths. (The designer must guarantee structural CDC synchronizers are in place via SpyGlass CDC).

Q131: 1-Deep Async FIFO Glitch and Throughput Collapse

ADVANCED 60%

Why is an Asynchronous FIFO of Depth = 1 impossible to build reliably using standard Gray pointer synchronization?

- A. Depth 1 requires negative bits.
- B. Pointer synchronization requires at least 2 flip-flop delays ($2 \times T_{dest_clk}$); in a 1-deep FIFO, the Full flag cannot clear in time to allow back-to-back writes, reducing throughput to near zero, and Gray pointers cannot distinguish empty from full with a 1-bit counter.
- C. Dual-port RAMs do not support 1 address location.
- D. Synthesis tools optimize Depth 1 into an inverter.

Detailed Solution & Technical Explanation

Correct Answer: B

Gray code pointer comparison requires at least 2 addresses ($N + 1$ pointer bits) to differentiate full from empty states. Furthermore, synchronizer latency requires a minimum depth (typically ≥ 4 or 8 words) to sustain continuous pipelined throughput.

Q132: Combinational Glitches on Synchronizer Inputs

ADVANCED 60%

If combinational logic is placed between the transmitting register and the input of a 2-FF synchronizer in an asynchronous clock domain crossing: Why is this an extreme CDC violation?

- A. It increases cell area.
- B. Combinational logic generates dynamic hazards and intermediate glitches; if an asynchronous clock edge samples during an intermediate glitch, the synchronizer will capture and propagate a false pulse to the destination domain.
- C. The synthesis tool removes combinational logic on CDC paths.
- D. It inverts clock polarity.

Detailed Solution & Technical Explanation

Correct Answer: B

Inputs to synchronizers must be **clean registered outputs** directly from flip-flop Q pins. Combinational hazards (glitches) that would normally settle harmlessly before a synchronous clock edge can be captured by an asynchronous clock edge, causing catastrophic false transitions.

Q133: Quasi-Static Signals Handling in CDC Verification

ADVANCED 60%

What is a **Quasi-Static Signal** in CDC verification, and how is it handled?

- A. A signal that toggles at 10 GHz.
- B. A software configuration signal written during system initialization and held constant during normal operation; it does not require a 2-FF synchronizer, but must be formally constrained/waived in SpyGlass CDC as `quasi_static`.
- C. A clock signal generated by an RC oscillator.
- D. A defective floating net.

Detailed Solution & Technical Explanation

Correct Answer: B

Quasi-static signals (e.g., debug mode enable, baud rate configuration divisor) never switch during active datapath transactions. They do not need hardware synchronizers, but must be declared to CDC linting tools to suppress false violation reports.

Q134: Async FIFO Read-to-Write Pipelined Latency

ADVANCED 60%

In an Async FIFO, when the read pointer is synchronized to the write clock domain, how many write clock cycles minimum does it take for the write domain to realize that a read has occurred?

- A. 0 cycles
- B. Exactly 1 cycle
- C. 2 to 3 write clock cycles (due to the 2-FF synchronizer stages plus clock phase alignment)
- D. 100 cycles

Detailed Solution & Technical Explanation

Correct Answer: C

The Gray read pointer must pass through the 2-stage synchronizer clocked by `wr_clk`. Depending on when the read pointer changes relative to the next `wr_clk` active edge, the update is registered after 2 to 3 full `wr_clk` periods.

Q135: Toggle Synchronizer Architecture Protocol

ADVANCED 60%

How does a **Toggle Synchronizer** successfully convey a 1-cycle pulse from a fast clock domain into a slow clock domain?

- A. It amplifies the voltage of the pulse.
- B. In the fast domain, every incoming pulse inverts (toggles) the state of a T-flip-flop (0 → 1 or 1 → 0); the continuous level transition is safely synchronized via a 2-FF synchronizer in the slow domain, where an XOR edge detector recreates a 1-cycle pulse.
- C. It stores the pulse in an analog capacitor.
- D. It converts the pulse into a PWM waveform.

Detailed Solution & Technical Explanation

Correct Answer: B

Converting a narrow transient pulse into a permanent level toggle converts the high-frequency event into a static DC level. The destination domain synchronizes the level and uses an XOR gate ($Q_{sync} \oplus Q_{sync_delayed}$) to regenerate a clean 1-cycle destination pulse.

Q136: SDC `set_max_delay -datapath_only` on CDC Nets

ADVANCED 60%

Why is `set_max_delay -datapath_only` specified on asynchronous CDC crossing paths instead of a standard `set_max_delay`?

- A. To force the tool to place all cells in the center of the chip.
- B. `-datapath_only` ignores clock skew and source/capture clock latencies (which are meaningless between asynchronous clocks) and strictly constrains the maximum routing wire delay across the asynchronous boundary to prevent excessive bit skew.
- C. To disable power analysis.
- D. To convert flip-flops into latches.

Detailed Solution & Technical Explanation

Correct Answer: B

Standard `set_max_delay` includes clock arrival times, which are undefined between asynchronous clocks. Specifying `-datapath_only` instructs STA to bound purely the combinational and routing propagation delay between the domains, keeping bus skew within safe multi-bit margins.

8 Formal Verification & SystemVerilog Assertions (SVA)

Q137: Logic Equivalence Checking (LEC) Goal

EASY 10%

What is the primary objective of Logic Equivalence Checking (e.g. Synopsys Formality, Cadence Conformal)?

- A. To measure dynamic power dissipation during functional simulation.
- B. To mathematically prove that an **Implementation Netlist** (e.g. Synthesized or PnR netlist) has identical Boolean functionality to the **Reference Design** (e.g. Golden RTL) for all possible input combinations without applying test vectors.
- C. To calculate the physical silicon yield.
- D. To test for stuck-at physical fabrication defects.

Detailed Solution & Technical Explanation

Correct Answer: B

Formal LEC is an exhaustive mathematical proof comparing two design representations (RTL vs Netlist, or Pre-PnR vs Post-PnR Netlist). It divides logic into combinatorial cones between register/port compare points and proves Boolean equivalence across all 2^N input states without running testbenches.

Q138: Formality Compare Points and Logic Cones

MODERATE 30%

In Synopsys Formality, what structures are automatically extracted as **Compare Points** between Reference and Implementation designs?

- A. Power ground pads.
- B. Primary Inputs, Primary Outputs, Sequential Elements (Flip-Flops / Latches), and Black Boxes.
- C. Internal wire names.
- D. Clock buffers only.

Detailed Solution & Technical Explanation

Correct Answer: B

LEC tools break complex circuits into independent logic cones bounded by **Compare Points**: Primary Inputs, Primary Outputs, Flip-Flops, Latches, and Black Box boundaries. Equivalence is proven independently for each logic cone.

Q139: Immediate vs Concurrent SystemVerilog Assertions

MODERATE 30%

What is the key distinction between **Immediate Assertions** and **Concurrent Assertions** in SystemVerilog Assertions (SVA)?

- A. Immediate assertions consume time; Concurrent assertions do not.
- B. **Immediate assertions** (`assert (a == b)`) evaluate combinational in the active region like procedural `if` statements and are prone to simulation glitches; **Concurrent assertions** (`assert property (@(posedge clk) ...)`) are clock-driven, multi-cycle temporal checks evaluated in the **Observed** simulation region.
- C. Concurrent assertions can only be used in C++ testbenches.
- D. Immediate assertions are only evaluated at time 0.

Detailed Solution & Technical Explanation

Correct Answer: B

Immediate assertions execute procedurally within simulation blocks and can trigger false glitch warnings

during intra-cycle transitions. Concurrent assertions sample signals deterministically in the Preponed region and evaluate temporal properties over multiple clock cycles in the Observed region.

Q140: SVA Implication Operators ($\rightarrow$ vs $\Rightarrow$)

MODERATE 30%

In SystemVerilog Assertions, what is the operational difference between `req  $\rightarrow$  ack` and `req  $\Rightarrow$  ack`?

- A. $\rightarrow$ checks `ack` in the next clock cycle; $\Rightarrow$ checks `ack` in the same clock cycle.
- B. **Overlapped ($\rightarrow$):** If `req` is true on clock cycle N , `ack` must be true on the **same clock cycle N** ; **Non-Overlapped ($\Rightarrow$):** If `req` is true on clock cycle N , `ack` must be true on the **subsequent clock cycle $N + 1$** (equivalent to `req ##1 ack`).
- C. Both operators are completely identical.
- D. $\Rightarrow$ is an arithmetic assignment operator.

Detailed Solution & Technical Explanation

Correct Answer: B

The overlapped implication operator $\rightarrow$ evaluates the consequent in the same cycle that the antecedent succeeds. The non-overlapped operator $\Rightarrow$ evaluates the consequent exactly one clock cycle later.

Q141: Resolving Unmapped Points in LEC with SVF Files

MODERATE 30%

What causes **Unmapped Points** during a Formality LEC run, and what is the primary diagnosis step?

- A. All standard cells are missing from the technology library.
- B. Registers were renamed during synthesis (e.g. Register Retiming, FSM re-encoding, or unused register removal); the engineer must load a `.svf` (Synchronized Verification Format) guidance file from Design Compiler to provide name-mapping hints.
- C. The clock frequency was set too high.
- D. The netlist has too many power domains.

Detailed Solution & Technical Explanation

Correct Answer: B

When synthesis performs aggressive optimizations (state encoding, register merging, boundary optimization, retiming), register names change. If Formality cannot automatically pair reference registers with implementation registers, it reports Unmapped Points. Loading the `.svf` file supplies transformation records that guide point alignment.

Q142: SVA `$past` and `$rose` Sampled Value Functions

ADVANCED 60%

Consider the following SVA property:

```
property p_check_ack;
    @(posedge clk) disable iff (!rst_n)
        $rose(ack)  $\rightarrow$  ($past(req, 2) == 1'b1);
endproperty
assert property (p_check_ack);
```

What functional behavior does this assertion mathematically enforce?

- A. Whenever `ack` is low, `req` must be high 2 cycles later.
- B. Whenever `ack` transitions from 0 to 1 (rising edge), the signal `req` must have been high exactly 2 clock cycles prior.
- C. `ack` and `req` must toggle simultaneously every 2 cycles.

D. `req` must remain high for 2 consecutive cycles after `ack` falls.

Detailed Solution & Technical Explanation

Correct Answer: B

`$rose(ack)` detects a 0-to-1 transition on `ack`. The antecedent triggers on this edge, requiring `$past(req, 2)` (the value of `req` sampled 2 clock cycles in the past) to equal 1'b1.

Q143: SVA Consecutive Repetition Operator [*n]

ADVANCED 60%

What does the SVA sequence `a ##1 b [*3] ##1 c` verify?

- A. Signal `a` is high, followed 1 cycle later by signal `b` being high for **3 consecutive clock cycles**, followed 1 cycle later by signal `c` being high.
- B. Signals `a`, `b`, and `c` are multiplied together.
- C. Signal `b` toggles 3 times in 1 cycle.
- D. Signal `a` is repeated 3 times.

Detailed Solution & Technical Explanation

Correct Answer: A

`b [*3]` is the consecutive repetition operator, denoting that condition `b` must hold true for 3 successive clock cycles: `b ##1 b ##1 b`.

Q144: SVA Non-Consecutive [=n] vs Goto [->n] Repetition

ADVANCED 60%

What is the difference between Goto Repetition `a [->3]` and Non-Consecutive Repetition `a [=3]`?

- A. `[->3]` requires `a` to occur on consecutive cycles; `[=3]` does not.
- B. **Goto Repetition (`a [->3]`)**: The sequence completes on the **exact clock cycle** where `a` is true for the 3rd time; **Non-Consecutive Repetition (`a [=3]`)**: The sequence matches when `a` has occurred 3 times and can continue to match on subsequent cycles as long as `a` remains false before the next event.
- C. Both are identical in formal verification.
- D. `[=3]` is only used in testbench scoreboards.

Detailed Solution & Technical Explanation

Correct Answer: B

`[->1]` (goto) matches at the precise cycle where the boolean becomes true (blocking further advancement until the match cycle). `[=1]` allows arbitrary empty cycles after the match has occurred before the next sequence element starts.

Q145: Bounded Model Checking (BMC) Bound Depth Meaning

ADVANCED 60%

In Formal Property Verification, what does it mean when a tool reports that an assertion is **Proven up to Bound $K = 50$ (BMC)**?

- A. The property is mathematically proven for infinite time ($K = \infty$).
- B. The tool mathematically proved that no bug can violate the assertion within 50 clock cycles from reset; however, a bug may still exist at cycle 51 or beyond unless a full mathematical induction proof (k-induction) is completed.
- C. The testbench ran 50 random vectors.
- D. The assertion was skipped.

Detailed Solution & Technical Explanation**Correct Answer: B**

Bounded Model Checking (BMC) unrolls the transition relation for K steps. It guarantees no counterexample exists within depth K , but does not guarantee correctness for all time. A complete mathematical proof requires unbounded formal proof engines (e.g. Craig Interpolation, PDR/IC3, or Induction).

Q146: Formal Verification Vacuity and cover property Checks**ADVANCED 60%**

An engineer writes the assertion: `assert property (@(posedge clk) req | => ack);`. During formal verification, the tool reports the assertion as **Passed**, but the design has a fatal bug where `req` is permanently tied to 0. What happened, and how is it prevented?

- A. The formal tool has a bug.
- B. The assertion passed **vacuously** (because the antecedent `req` was never true, the implication `| =>` is automatically satisfied without ever checking `ack`); it must be accompanied by `cover property (@(posedge clk) req)` to prove that the antecedent was actively exercised.
- C. The formal tool automatically disables false assertions.
- D. Formal verification cannot detect tied signals.

Detailed Solution & Technical Explanation**Correct Answer: B**

In formal implication ($A \implies B$), if A is false, the mathematical statement is true by definition (vacuous pass). To ensure assertions provide meaningful coverage, engineers write companion `cover property` checks to guarantee the antecedent triggers during verification.

Q147: assume property Over-Constraining Risks**ADVANCED 60%**

In Formal Property Verification, what is the danger of writing an over-constrained `assume property` on input signals?

- A. It increases simulation runtime by $10\times$.
- B. It can mask real design bugs (**Proof Vacuity / Over-Constraining**) by illegally restricting the formal engine from exploring valid real-world input stimulus scenarios.
- C. It causes synthesis to insert extra hardware.
- D. It turns the formal tool into an ATPG tool.

Detailed Solution & Technical Explanation**Correct Answer: B**

`assume` constraints limit the input state space explored by the formal solver. If an assumption is too restrictive (e.g. assuming an input can never arrive during a specific state), the formal tool will skip exploring those states, hiding genuine silicon bugs.

Q148: SVA disable iff Construct During System Reset**ADVANCED 60%**

Why must concurrent assertions always include `disable iff (!rst_n)`?

- A. To save memory during compilation.
- B. During active reset, flip-flops are clearing and intermediate signals are invalid; `disable iff (!rst_n)` immediately disables property evaluation while reset is asserted, preventing false assertion failures during system initialization.
- C. To allow assertions to run during scan shift mode.
- D. Because SVA does not support reset signals otherwise.

Detailed Solution & Technical Explanation**Correct Answer: B**

While reset is asserted, signals are indeterminate or in transition. Without `disable iff (!rst_n)`, assertions would evaluate during reset, generating hundreds of false violation reports.

Q149: Formality Solver Aborts on Hardware Multipliers**ADVANCED 60%**

When Formality encounters a complex 64×64 hardware multiplier, the compare points often status as **ABORT** / **TIMEOUT**. Why does this happen, and what is the standard formal solution?

- A. Multipliers have high dynamic power.
- B. Multiplier Boolean equivalence creates an exponential explosion in Binary Decision Diagrams (BDD) and SAT-solver clauses; the solution is to Black-Box the multiplier architecture in both Reference and Implementation or verify the multiplier hierarchy independently.
- C. Formality cannot parse multiplication operators.
- D. The clock period is too short.

Detailed Solution & Technical Explanation**Correct Answer: B**

Formal equivalence of large non-linear arithmetic structures (multipliers, dividers) is NP-hard. BDD sizes grow exponentially with operand width ($O(2^N)$). Designers verify multipliers hierarchically or isolate them as black-boxes for system-level LEC.

Q150: SVA \$stable Handshake Data Preservation**ADVANCED 60%**

Which SVA assertion guarantees that once `valid` is asserted high, the 32-bit `data` bus must remain strictly unchanged until `ready` is asserted high?

- A. `assert property (@(posedge clk) (valid && !ready) | => $stable(data));`
- B. `assert property (@(posedge clk) valid |-> $changed(data));`
- C. `assert property (@(posedge clk) data == ready);`
- D. `assert property (@(posedge clk) ready | => $past(data));`

Detailed Solution & Technical Explanation**Correct Answer: A**

In standard AXI/Ready-Valid protocols, a sender cannot change data once offered until accepted. If `valid` is high and `ready` is low on cycle N , on the next cycle ($| =>$) `data` must satisfy `$stable(data)` ($data_{current} == data_{previous}$).

9 Design for Testability (DFT, ATPG & JTAG)

Q151: Primary Goal of DFT in ASIC Manufacturing**EASY 10%**

What is the primary objective of Design for Testability (DFT) in digital ASIC engineering?

- A. To verify that the RTL meets functional specifications.
- B. To detect physical manufacturing defects (e.g. Silicon shorts, opens, metal bridges, transistor defects) in post-fabrication packaged chips using automated test equipment (ATE).
- C. To eliminate power dissipation in standby mode.
- D. To measure the maximum operating frequency of the PLL.

Detailed Solution & Technical Explanation**Correct Answer: B**

DFT does not verify functional design intent; rather, it provides hardware structures (Scan Chains, BIST, Boundary Scan) that allow Automated Test Equipment (ATE) to detect physical semiconductor manufacturing flaws with high fault coverage.

Q152: Stuck-At-0 (SA0) Fault Physical Modeling

EASY 10%

In structural fault modeling, what does a **Stuck-At-0 (SA0)** fault represent?

- A. A clock operating at zero frequency.
- B. A physical defect where a circuit node or transistor pin is permanently shorted to ground (V_{SS} / logic 0), regardless of input stimuli.
- C. A register that resets to 1.
- D. An open circuit on a power pin.

Detailed Solution & Technical Explanation**Correct Answer: B**

The classical Stuck-At Fault model abstracts physical manufacturing defects (such as bridging to ground or open gate connections) by modeling internal circuit nodes as permanently tied to logic 0 (SA0) or logic 1 (SA1).

Q153: Muxed-D Scan Flip-Flop Architecture

MODERATE 30%

How does a standard **Muxed-D Scan Flip-Flop** modify a conventional D flip-flop?

- A. It adds an extra clock input for high-speed operation.
- B. It places a 2-to-1 multiplexer on the D input, controlled by a **Scan_Enable (SE)** signal to select between functional data (D) when **SE=0** and scan shift test data (SI) when **SE=1**.
- C. It converts the flip-flop into a latch.
- D. It adds an integrated level shifter.

Detailed Solution & Technical Explanation**Correct Answer: B**

In Muxed-D scan architecture, setting **SE=1** connects all flip-flops into a serial shift register (SI to SO). Setting **SE=0** allows flip-flops to capture normal functional combinational logic outputs.

Q154: ATPG Scan Shift vs Capture Operational Sequence

MODERATE 30%

What is the correct operational sequence during a standard ATPG scan test cycle on ATE?

- A. Capture → Shift → Reset
- B. **Scan Shift Mode (SE=1)**: Shift test stimulus pattern into scan chains across N clock cycles → **Capture Mode (SE=0)**: Apply 1 functional clock pulse to capture combinational responses into flip-flops → **Scan Shift Mode (SE=1)**: Shift out captured responses for signature comparison while shifting in the next pattern.
- C. Burn-in → Functional Execution → Power Off
- D. Shift → Continuous Free-Running Clocks for 1 hour

Detailed Solution & Technical Explanation**Correct Answer: B**

Scan testing converts the sequential testing problem into a simple combinational test: stimulus is shifted

in serially (SE=1), captured through combinational logic in 1 cycle (SE=0), and shifted out for comparison (SE=1).

Q155: Fault Coverage vs Test Coverage Mathematical Calculation

MODERATE 30%

Given a design with:

- Total Faults = 10,000
- Detected Faults = 9,500
- Undetectable Faults (Tied/Redundant logic) = 500

What are the **Fault Coverage** and **Test Coverage** percentages?

- A. Fault Coverage = 95%; Test Coverage = 100%
- B. Fault Coverage = 90%; Test Coverage = 95%
- C. Fault Coverage = $\frac{9500}{10000} = 95.0\%$; Test Coverage = $\frac{9500}{10000-500} = \frac{9500}{9500} = 100.0\%$
- D. Both are identically 95%.

Detailed Solution & Technical Explanation

Correct Answer: C

- Fault Coverage = $\frac{\text{Detected Faults}}{\text{Total Faults}} = \frac{9500}{10000} = 95.0\%$.
- Test Coverage = $\frac{\text{Detected Faults}}{\text{Total Faults} - \text{Undetectable Faults}} = \frac{9500}{9500} = 100.0\%$.

Test coverage measures the efficiency of the ATPG tool against physically testable faults.

Q156: Boundary Scan / JTAG (IEEE 1149.1) Dedicated Pins

MODERATE 30%

What are the 4 mandatory dedicated test pins required by the IEEE 1149.1 JTAG Boundary Scan Standard?

- A. CLK, RST, DIN, DOUT
- B. TCK (Test Clock), TMS (Test Mode Select), TDI (Test Data In), TDO (Test Data Out) — with optional TRST* (Test Reset).
- C. VREF, VDD, VSS, SCAN_EN
- D. SCL, SDA, INT, READY

Detailed Solution & Technical Explanation

Correct Answer: B

IEEE 1149.1 defines a 4-wire standard interface: TCK (clock), TMS (state machine control), TDI (serial data in), and TDO (serial data out), with an optional active-low asynchronous reset TRST*.

Q157: JTAG TAP Controller 16-State FSM Reset Mechanism

MODERATE 30%

The JTAG TAP (Test Access Port) Controller state machine advances its states on which clock edge, and how is it reset to **Test-Logic-Reset** without using TRST*?

- A. Falling edge of TCK; by pulling TDI low for 5 cycles.
- B. Rising edge of TCK; by holding TMS = 1 for at least 5 consecutive TCK clock cycles, guaranteeing return to **Test-Logic-Reset** from any of the 16 states.
- C. Both edges of TCK; by powering down the chip.
- D. High level on TDO.

Detailed Solution & Technical Explanation

Correct Answer: B

The TAP controller state diagram is structured such that regardless of the starting state, holding $TMS=1$ transitions the state machine back to **Test-Logic-Reset** within a maximum of 5 rising edges of TCK.

Q158: At-Speed Transition Delay Fault Testing (LOC vs LOS)**ADVANCED 60%**

What is the difference between **Launch-on-Shift (LOS)** and **Launch-on-Capture (LOC / Broadside)** in at-speed transition fault testing?

- A. LOS uses slow clock frequencies; LOC uses DC voltages.
- B. **LOS (Launch-on-Shift)**: The test transition is launched on the last shift pulse (SE must switch from 1 to 0 at rated gigahertz speed, requiring a timing-critical scan-enable tree); **LOC (Launch-on-Capture)**: The transition is launched by a functional clock pulse while $SE=0$ (relaxed SE routing, but requires sequential ATPG).
- C. LOC does not support stuck-at faults.
- D. LOS requires analog test equipment.

Detailed Solution & Technical Explanation**Correct Answer: B**

At-speed transition testing requires two rated clock pulses: Launch and Capture. In LOS, the launch occurs on the final shift clock edge, requiring `Scan_Enable` to deassert in less than one clock period (T_{clk}). LOC applies both pulses during functional capture mode ($SE=0$), eliminating the need for an at-speed `Scan_Enable` network.

Q159: Scan Lockup Latches Across Clock Domains**ADVANCED 60%**

Why must a **Scan Lockup Latch (Active-Low Latch)** be inserted at the boundary where a scan chain crosses from Domain A (CLK_A) to Domain B (CLK_B) or between different clock edges?

- A. To invert the scan data.
- B. To eliminate **Hold Violations during Scan Shift Mode** caused by clock skew between the two domains; the negative-level latch holds data stable during the clock high phase, ensuring safe transfer even with severe inter-domain skew.
- C. To double the scan shift frequency.
- D. To compress ATPG test patterns.

Detailed Solution & Technical Explanation**Correct Answer: B**

During scan shift, flip-flops are connected in a direct shift chain with zero combinational logic ($T_{comb} = 0$). Any positive clock skew between registers causes immediate hold violations. An active-low lockup latch holds the output of the transmitter during the first half-cycle, providing a half-period of hold margin.

Q160: Memory BIST (MBIST) and March Algorithms**ADVANCED 60%**

Why are embedded SRAMs tested using **MBIST** engines running **March Algorithms** (e.g. **March C-**) rather than standard scan ATPG chains?

- A. Memory cells cannot be modeled with gate-level stuck-at models; March tests systematically write and read patterned sequences ($O(N)$ complexity) to detect complex memory-specific physical defects like transition faults, address decoder open faults, coupling faults between adjacent cells, and retention faults.
- B. Standard scan flip-flops cannot be fabricated inside SRAMs.
- C. MBIST requires zero silicon area.

D. Both A and B are physically correct.

Detailed Solution & Technical Explanation

Correct Answer: D

Embedded memories have dense transistor arrays that cannot accommodate internal scan chains. Dedicated hardware MBIST controllers run algorithmic March test patterns (e.g., $\uparrow (w0), \uparrow (r0, w1), \downarrow (r1, w0)$) to detect cell coupling, address decoder, and pattern-sensitive defects.

Q161: Logic BIST (LBIST): PRPG and MISR Architecture

ADVANCED 60%

In an autonomous Logic BIST (LBIST) architecture: What are the roles of the **PRPG** and the **MISR**?

- A. PRPG measures temperature; MISR controls voltage.
- B. **PRPG (Pseudo-Random Pattern Generator)**: An LFSR that autonomously generates pseudo-random test stimulus vectors for scan chains; **MISR (Multiple-Input Signature Register)**: An LFSR/XOR compressor that compresses the captured output responses into a compact multi-bit signature for pass/fail comparison.
- C. PRPG is an external ATE pin; MISR is an internal clock divider.
- D. They replace the CPU instruction cache.

Detailed Solution & Technical Explanation

Correct Answer: B

LBIST enables on-chip self-testing (critical for automotive ISO 26262 safety): a PRPG generates millions of pseudorandom patterns on-chip, and a MISR compresses output states into a final 32/64-bit signature compared against a golden ROM value.

Q162: Scan Compression (EDT / TestKompress)

ADVANCED 60%

How does **Embedded Deterministic Test (Scan Compression)** achieve 50× reduction in test time and ATE data volume?

- A. By dropping 50
- B. An on-chip **Decompressor** expands a small number of external ATE input channels into hundreds of internal short scan chains, while an on-chip **Compactor** compresses the hundreds of internal output chains back into a few ATE output pins, utilizing the fact that > 95% of ATPG bits are don't-cares (Xs).
- C. By increasing the ATE clock frequency to 100 GHz.
- D. By testing only 1 chip per wafer.

Detailed Solution & Technical Explanation

Correct Answer: B

In standard ATPG, > 95% of test bits are "don't cares" (X) needed only to propagate a few targeted faults. Scan compression uses linear feedback decompressors to broadcast deterministic values to internal scan chains, dramatically reducing test time and tester memory.

Q163: JTAG BYPASS Instruction Functionality

ADVANCED 60%

What is the function of the mandatory JTAG instruction **BYPASS**?

- A. It disconnects the chip from the circuit board.
- B. It connects TDI directly to TDO through a single **1-bit Bypass Register** (with 1 cycle latency), allowing test data to bypass this IC quickly when testing other chips on the same PCB scan chain.

- C. It bypasses all setup timing checks in the chip.
- D. It bypasses the chip's internal voltage regulators.

Detailed Solution & Technical Explanation

Correct Answer: B

When multiple chips share a JTAG chain on a PCB, testing a single target chip would be slowed down by shifting through hundreds of boundary scan cells in unrelated ICs. Setting unrelated chips to **BYPASS** shortens their internal shift path to a single 1-bit register.

Q164: Scan Chain Balancing Rationale

ADVANCED 60%

Why must multiple internal scan chains be balanced to equal lengths (e.g. 10 chains of 500 flip-flops each, rather than 1 chain of 4500 and 9 of 50)?

- A. To balance the dynamic power across package pins.
- B. The total shift time for each test pattern is bounded by the **longest single scan chain**; unbalanced chains force shorter chains to be padded with dummy bits, wasting tester memory and dramatically increasing test time.
- C. Unbalanced chains cause clock skew during functional mode.
- D. Synthesis tools cannot route unbalanced chains.

Detailed Solution & Technical Explanation

Correct Answer: B

All scan chains shift simultaneously during test. The ATE must supply shift cycles equal to the longest chain length (L_{max}). If one chain is 4500 bits and others are 50 bits, the ATE must execute 4500 shift cycles per pattern, wasting 90

Q165: Undetectable Faults and Redundant Logic

ADVANCED 60%

Consider the logic function $Y = A \cdot B + \bar{A} \cdot C + B \cdot C$. Why is the Stuck-At-0 fault on the input to term $B \cdot C$ classified by ATPG as an **Undetectable / Redundant Fault**?

- A. Because the cell is not connected to clock.
- B. The term $B \cdot C$ is a consensus term (algebraically redundant); no input stimulus vector can ever propagate the effect of a fault on this gate to primary output Y without being masked by $A \cdot B$ or $\bar{A} \cdot C$.
- C. Because ATPG tools do not support 3-input equations.
- D. Because the output is always zero.

Detailed Solution & Technical Explanation

Correct Answer: B

Redundant logic cannot be tested because its output can never uniquely control the circuit output independently of other paths. Redundant gates reduce test coverage metrics and waste area, though they are sometimes intentionally used to prevent glitches.

Q166: X-Bounding and X-Masking in Scan Compactors

ADVANCED 60%

Why are uninitialized memory outputs, analog IP blocks, or asynchronous crossings hazardous to on-chip scan output compactors (MISRs), and how is this resolved?

- A. They consume excess static current.
- B. Unknown logic states (Xs) corrupt the entire MISR signature calculation (since $X \oplus 0 = X$), destroying test validity; they must be blocked using **X-Bounding / X-Masking logic** before reaching the compactor.

- C. They cause physical short-circuits in the scan chains.
- D. They disable the JTAG TAP controller.

Detailed Solution & Technical Explanation

Correct Answer: B

Compaction structures (MISRs or XOR trees) combine hundreds of bits. If even a single bit is X (unknown), the unknown state propagates through XOR gates and corrupts the entire signature. X-masking logic selectively forces known values onto suspect scan channels.

Q167: IDDQ Current Testing Principles

ADVANCED 60%

What is the principle of I_{DDQ} **Testing**, and what physical defects does it detect?

- A. It tests dynamic switching power at 5 GHz.
- B. It measures the quiescent steady-state supply current (I_{DDQ}) when all internal nodes are static; defective circuits with bridging shorts or gate-oxide breakdowns draw orders-of-magnitude higher quiescent current than nominal leakage.
- C. It tests radiation hardness in aerospace chips.
- D. It checks the resistance of bonding wires.

Detailed Solution & Technical Explanation

Correct Answer: B

In a static CMOS circuit, steady-state current is purely leakage. If a bridging defect exists between two nodes driven to opposite values, a low-impedance short-circuit path from V_{DD} to V_{SS} draws massive current ($> 100 \mu\text{A}$), easily flagged by I_{DDQ} monitoring.

Q168: IEEE 1500 Standard for Embedded Core Test

ADVANCED 60%

What is the primary role of the **IEEE 1500 Standard** in modern multi-core SoC design?

- A. It provides high-speed PCIe communication.
- B. It defines an embedded modular test wrapper (Wrapper Instruction Register - WIR, Wrapper Boundary Cells - WBC) around individual reusable IP cores, enabling isolated modular testing of cores and SoC interconnects.
- C. It standardizes the USB protocol.
- D. It defines FPGA bitstream formats.

Detailed Solution & Technical Explanation

Correct Answer: B

IEEE 1500 provides a standardized wrapper around complex modular IP cores (CPUs, GPUs, DSPs). It allows each core to be tested independently (Internal Test) and allows the surrounding interconnect between cores to be tested (External Test).

10 Gate-Level Simulation (GLS) & SDF

Q169: Zero-Delay vs SDF-Annotated GLS

EASY 10%

What is the difference between a **Zero-Delay Gate-Level Simulation** and a **Full-Timing SDF GLS**?

- A. Zero-delay runs on hardware emulators; SDF runs in SPICE.
- B. Zero-delay simulation evaluates logic gate functions with zero propagation delay across all gates and wires (used for initial functional netlist sanity checks); SDF GLS back-annotates

precise cell delays, wire delays, and setup/hold timing checks from Standard Delay Format (.sdf) files.

- C. Zero-delay GLS includes crosstalk; SDF does not.
- D. There is no difference.

Detailed Solution & Technical Explanation

Correct Answer: B

Zero-delay netlist simulation verifies that synthesis did not corrupt functional logic expressions. SDF back-annotated simulation uses physical timing data to verify dynamic operation, clock gating, asynchronous resets, and timing-dependent interfaces under real delays.

Q170: Verilog \$sdf_annotate System Task

MODERATE 30%

In a Verilog testbench, what does the following system task execute?

```
$sdf_annotate("top_layout.sdf", u_dut, , "sdf.log", "MAXIMUM");
```

- A. It compiles the testbench into C++ code.
- B. It reads the timing delay file `top_layout.sdf` and annotates the **Maximum** timing delays (slow-corner delays) onto the instantiated hierarchy `u_dut`, logging errors to `sdf.log`.
- C. It disables all timing checks inside `u_dut`.
- D. It measures dynamic power dissipation during simulation.

Detailed Solution & Technical Explanation

Correct Answer: B

`$sdf_annotate` is the standard IEEE 1497 Verilog system task that parses .sdf files and back-annotates IOPATH cell delays, INTERCONNECT net delays, and `$setup/$hold` timing checks into the simulator's timing engine.

Q171: Simulator Response to \$setup and \$hold Violations

MODERATE 30%

What occurs in an SDF-annotated gate-level simulation when a data transition violates the `$setup` timing check specified inside a standard cell's `specify` block?

- A. The simulation immediately aborts with a segmentation fault.
- B. The simulator issues a timing violation warning in the console and drives the output pin Q of the affected flip-flop to an **unknown state (X)** to model non-deterministic behavior.
- C. The simulator ignores the violation and continues normally.
- D. The clock frequency is automatically halved.

Detailed Solution & Technical Explanation

Correct Answer: B

Standard cell Verilog models define `specify` blocks with `$setup`, `$hold`, `$recovery`, and `$removal` checks. When violated, the library cell model's notifier register toggles, causing the output Q to corrupt to X (unknown state).

Q172: Why GLS is Mandatory Alongside STA and LEC Signoff

MODERATE 30%

If Static Timing Analysis (STA) exhaustively proves timing closure and Formality (LEC) proves functional equivalence, why is Gate-Level Simulation (GLS) still required before tapeout?

- A. To check that standard cell colors look correct.
- B. To verify asynchronous initialization sequences, reset deassertion recovery/removal be-

havior, dynamic clock switch behaviors, PLL lock sequences, and power-up/down UPF isolation sequences that cannot be fully verified by static analysis.

- C. Because STA tools cannot measure setup slack.
- D. To replace physical DRC checks.

Detailed Solution & Technical Explanation

Correct Answer: B

STA is static and relies on design constraints (which may contain incorrect false path or case analysis waivers). GLS provides dynamic verification of multi-clock synchronization, power-up reset release, and complex mixed-signal handshakes in real time.

Q173: Simulation Arguments: `+notimingchecks` vs `+nospecify`

MODERATE 30%

What is the effect of running a gate-level simulator with the argument `+notimingchecks`?

- A. It deletes the gate-level netlist.
- B. It executes the simulation using the cell and interconnect propagation delays, but disables all `$setup`, `$hold`, `$recovery`, and `$width` timing checks, preventing flip-flops from corrupting to X.
- C. It converts all clocks to zero delay.
- D. It bypasses all assertion checks.

Detailed Solution & Technical Explanation

Correct Answer: B

`+notimingchecks` is commonly used during initial reset power-up phases to prevent uninitialized asynchronous reset assertions from generating cascades of false X states across the chip before clocks stabilize.

Q174: X-Pessimism in Combinational MUX Gates

ADVANCED 60%

Consider a 2-to-1 MUX implemented with basic gates: `assign Y = (A & S) | (B & ~S);`. If `A = 1`, `B = 1`, and select line `S = X` (unknown): What does the gate-level simulator output for Y, and why is this **X-Pessimism**?

- A. Simulator outputs `Y = 1` (correct physical behavior).
- B. Simulator outputs `Y = X` (because `1 & X = X`, `1 & ~X = X`, and `X | X = X`), even though in real silicon Y is physically guaranteed to be 1 regardless of S.
- C. Simulator outputs `Y = 0`.
- D. Simulator crashes.

Detailed Solution & Technical Explanation

Correct Answer: B

4-state HDL simulators evaluate gates individually without symbolic correlation. In reality, whether `S = 0` or `S = 1`, `Y = 1`. The simulator evaluates each branch as X and their OR as X. This "X-Pessimism" causes false simulation lockups in GLS that do not exist in silicon.

Q175: Resolving X-Pessimism in GLS via Xprop

ADVANCED 60%

How do verification teams overcome catastrophic X-Pessimism during gate-level simulation?

- A. By deleting all multiplexers.
- B. Using specialized simulator flags (e.g. Synopsys VCS `+vcs+xprop` or Cadence X-Optimism modes), modifying standard cell simulation models to handle correlated conditions, and initializing memories/registers with random deterministic values during reset.

- C. By forcing all signals to 1.
- D. By disabling the clock tree.

Detailed Solution & Technical Explanation

Correct Answer: B

EDA tools provide X-propagation modes (**Xprop**) that analyze logic cones to determine if boolean outputs are independent of the unknown input, preventing artificial X propagation while preserving true uninitialized state tracking.

Q176: Glitch Filtering: Inertial vs Transport Delay Models

ADVANCED 60%

In gate-level SDF simulation, what is the difference between **Inertial Delay** and **Transport Delay**?

- A. Inertial delay applies only to wires; Transport delay applies only to gates.
- B. **Inertial Delay** (default in Verilog gates): Pulses shorter than the gate's propagation delay are rejected and filtered out as glitches; **Transport Delay** (**+transport_path_delays**): Propagates all pulses through wires and interconnects regardless of duration, accurately modeling narrow glitch propagation across physical routes.
- C. Transport delay doubles the pulse width.
- D. Inertial delay converts pulses into sine waves.

Detailed Solution & Technical Explanation

Correct Answer: B

Standard gate models use inertial delay, which suppresses pulses narrower than the cell propagation delay. Physical metal interconnects behave with transport delay (pure delay line). Running GLS with **+transport_path_delays** ensures narrow glitches on routing tracks reach downstream gates.

Q177: SDF IOPATH vs INTERCONNECT Delays

ADVANCED 60%

In an SDF file, what is the distinction between (**IOPATH in out (0.15)**) and (**INTERCONNECT u1/out u2/in (0.35)**)?

- A. IOPATH models wire delay; INTERCONNECT models transistor delay.
- B. **IOPATH** specifies the internal cell delay from an input pin to an output pin of a specific standard cell; **INTERCONNECT** specifies the net/wire propagation delay from a driving cell output pin to a load cell input pin across physical metal routes.
- C. INTERCONNECT is only used for analog pins.
- D. IOPATH is deprecated in IEEE 1497.

Detailed Solution & Technical Explanation

Correct Answer: B

SDF separates cell delay from interconnect delay. **IOPATH** records the internal switching delay of the standard cell arc, while **INTERCONNECT** records the distributed RC wire delay extracted from physical layout parasitics (SPEF).

Q178: Negative Setup and Hold Times in Characterized Cells

ADVANCED 60%

When an SDF file annotates a **Negative Hold Time** (e.g. (**HOLD D (posedge CK) (-0.05)**)) on a flip-flop: What does this physically mean, and how does the simulator handle it?

- A. The flip-flop operates backwards in time.
- B. An internal delay on the data path inside the standard cell exceeds the delay on the internal

clock path; data can physically change 0.05 ns *before* the external clock pin transition without corrupting the captured state.

- C. The flip-flop has a manufacturing defect.
- D. Negative numbers are illegal in SDF and cause simulation aborts.

Detailed Solution & Technical Explanation

Correct Answer: B

Standard cells are characterized at their external boundary pins. If internal buffering delays the data line more than the clock line, data does not need to be held until the external clock edge—it can change slightly before it. Simulators handle negative values using negative timing check algorithms (\$setuphold).

Q179: Pulse Rejection and Error Limits (+pulse_r / +pulse_e)

ADVANCED 60%

What is the purpose of specifying simulator arguments +pulse_r/0 and +pulse_e/100 during SDF gate-level simulation?

- A. To control the simulator's CPU thread count.
- B. To define pulse rejection thresholds: Any pulse with width < 0% of cell delay is rejected (ignored), while any pulse with width between 0% and 100% of cell delay is flagged as a hazard and driven to X (**Error**) to expose potential glitch corruption.
- C. To scale the clock period.
- D. To measure power dissipation.

Detailed Solution & Technical Explanation

Correct Answer: B

Pulse control options handle narrow transition spikes. Setting pulse_e/100 converts any partial pulse (glitch narrower than cell delay) into an unknown X state, highlighting potential race conditions in simulation waveforms.

Q180: Diagnostic Workflow for GLS Reset Glitches

ADVANCED 60%

During full-timing SDF GLS of a multi-million gate SoC, the entire chip corrupts to X within the first 100ns of simulation. What is the standard diagnostic workflow to identify the root cause?

- A. Immediately re-synthesize the design with higher optimization.
- B. Trace back the origin of the first X transition in the waveform viewer (e.g. Verdi / DVE), inspect timing violation logs at time T_0 to check for uninitialized flip-flops or recovery/removal violations on the asynchronous reset tree, and verify whether +notimingchecks was enabled during the reset asserting window.
- C. Increase the supply voltage in the testbench.
- D. Disable all clock generators.

Detailed Solution & Technical Explanation

Correct Answer: B

In GLS, a single flip-flop entering X rapidly cascades across the entire chip. Tracing back through the schematic/waveform to the very first signal that went to X reveals whether it was triggered by a real timing violation (e.g. reset recovery failure) or an uninitialized memory array.

11 Physical Design & Place & Route (PnR)

Q181: Floorplanning Core Utilization Calculation

EASY 10%

If a chip has a total standard cell area of 2.0 mm^2 and the designer defines a core area of 4.0 mm^2 : What is the **Core Utilization** factor?

- A. 25%
- B. 50%
- C. 75%
- D. 200%

Detailed Solution & Technical Explanation

Correct Answer: B

$$\text{Core Utilization} = \frac{\text{Total Cell Area}}{\text{Total Core Area}} = \frac{2.0 \text{ mm}^2}{4.0 \text{ mm}^2} = 0.50 = 50\%.$$

Q182: Sequential Stages of the Physical Design Flow

EASY 10%

What is the standard sequential execution flow of Physical Design (PnR)?

- A. Routing → Placement → Floorplanning → Tapeout
- B. Floorplanning / Power Planning → Standard Cell Placement → Clock Tree Synthesis (CTS) → Detail Routing → Physical Verification (DRC/LVS) & Signoff Extraction
- C. CTS → ATPG → Synthesis → Fabrication
- D. Packaging → Floorplanning → Extraction

Detailed Solution & Technical Explanation

Correct Answer: B

Physical design begins with die/core floorplanning and power grid construction, places standard cells, synthesizes balanced clock distribution trees (CTS), routes all signal nets, and finishes with signoff extraction (SPEF) and physical DRC/LVS verification.

Q183: Macro Placement Halos and Keepout Margins

MODERATE 30%

Why are **Placement Halos (Keepout Margins)** placed around large hard IP macros (e.g. SRAMs, PLLs) during floorplanning?

- A. To prevent the macro from consuming static power.
- B. To reserve physical routing and placement channels around the macro perimeter, preventing standard cells from being placed too close to macro pins, avoiding pin access congestion and noise coupling.
- C. To isolate analog grounds from digital grounds.
- D. To allow heat dissipation only.

Detailed Solution & Technical Explanation

Correct Answer: B

Placement halos create an exclusion boundary around memory blocks where no standard cells can be placed. This ensures adequate space for power routing rings and provides routing channels for the high density of pins entering and exiting the macro.

Q184: Static vs Dynamic IR Drop on Power Grids

MODERATE 30%

What is the distinction between **Static IR Drop** and **Dynamic IR Drop** on the on-chip power distribution network (PDN)?

- A. Static IR drop occurs in sleep mode; Dynamic occurs in test mode.
- B. **Static IR Drop** is the steady-state average DC voltage drop ($\Delta V = I_{avg} \times R_{grid}$) caused

by wire resistance; **Dynamic IR Drop** is the localized transient voltage drop caused by simultaneous high-frequency switching current surges ($\Delta V = I(t) \cdot R + L \cdot \frac{di}{dt}$) on active clock edges.

- C. Static IR drop only affects I/O pads.
- D. Dynamic IR drop can be eliminated by removing decoupling capacitors.

Detailed Solution & Technical Explanation

Correct Answer: B

Static IR drop depends on average DC current consumption and metal rail resistance. Dynamic IR drop is a high-frequency transient drop caused by thousands of flip-flops switching simultaneously on a clock edge ($L \cdot di/dt$).

Q185: Decoupling Capacitors (Decap Cells) Role

MODERATE 30%

How do **Decoupling Capacitor (Decap)** cells mitigate dynamic IR drop in high-speed digital designs?

- A. They disconnect the power supply during switching.
- B. They act as local high-frequency charge reservoirs placed close to switching logic cells; when large current surges occur on clock edges, Decap cells supply instantaneous charge locally, preventing power grid voltage droop.
- C. They increase wire resistance.
- D. They eliminate static leakage current.

Detailed Solution & Technical Explanation

Correct Answer: B

Standard power supply pads are too far away (high inductance) to supply instantaneous sub-nanosecond switching currents. Decap cells placed in empty floorplan sites store local charge and supply transient currents directly to neighboring gates.

Q186: Clock Tree Synthesis (CTS) Optimization Targets

MODERATE 30%

What are the primary goals of **Clock Tree Synthesis (CTS)**?

- A. To minimize cell area and remove all buffers.
- B. To build a balanced distribution network (using H-Tree, Clock Mesh, or Balanced Tree structures) that distributes the clock signal to millions of sequential elements with **Minimum Skew, Controlled Latency (Insertion Delay), and Symmetrical Transition Slews**.
- C. To convert flip-flops into scan chains.
- D. To eliminate dynamic power completely.

Detailed Solution & Technical Explanation

Correct Answer: B

CTS inserts balanced buffer/inverter trees to deliver clock edges to all flip-flops simultaneously, minimizing spatial clock skew ($T_{capture} - T_{launch}$), maintaining sharp rise/fall slews, and controlling total insertion delay.

Q187: Antenna Effect and Diode Shunt Insertion

ADVANCED 60%

During semiconductor fabrication, what is the **Antenna Effect**, and how is it resolved in Physical Design?

- A. Metal wires picking up 5G radio signals.
- B. Long exposed metal tracks act as antennas during plasma etching, collecting electrostatic charge; the accumulated charge discharges through thin MOSFET gate oxides, permanently destroying the transistor dielectric. **Fix:** Metal layer jumping (routing up to higher metals) or inserting **Antenna Diodes** near the gate to safely shunt charge to the substrate.
- C. Wires vibrating due to thermal heat.
- D. Clock trees radiating electromagnetic noise.

Detailed Solution & Technical Explanation

Correct Answer: B

During plasma etching, long metal routes accumulate charge. If connected to a transistor gate without a source/drain diffusion path to bleed charge, the voltage exceeds gate oxide breakdown limits. Inserting reverse-biased antenna diodes provides a low-impedance discharge path to ground.

Q188: Electromigration (EM) Physics and Multi-Cut Vias

ADVANCED 60%

What causes **Electromigration (EM)** in metal interconnects, and what are its physical consequences?

- A. High voltage causing dielectric breakdown.
- B. High directional direct-current density ($J > J_{max}$) causes moving electrons to physically transfer momentum to metal atoms ("electron wind"), moving metal ions over time to create **Voids (open circuits)** and **Hillocks/Whiskers (short circuits to adjacent tracks)**. **Fix:** Widen metal wires and insert multi-cut vias.
- C. Thermal expansion cracking the silicon substrate.
- D. Chemical corrosion from packaging materials.

Detailed Solution & Technical Explanation

Correct Answer: B

Electromigration is the physical transport of material in a conductor due to the momentum transfer between electrons and metal ions. Over time, high current densities create voids (leading to open circuits) or hillocks (leading to shorts). Widening wires reduces current density $J = I/A$.

Q189: Global Routing vs Detail Routing in PnR Tools

ADVANCED 60%

In PnR tools (Innovus / ICC2), what is the difference between **Global Routing** and **Detail Routing**?

- A. Global routing routes clocks; Detail routing routes resets.
- B. **Global Routing:** Partitions the core into global routing cells (G-cells) and plans coarse capacity/demand usage to identify congestion hotspots without placing actual metal shapes; **Detail Routing:** Instantiates physical metal tracks, assigns exact routing layers, and places vias to meet all foundry DRC rules.
- C. Detail routing runs before placement.
- D. Global routing is performed in SPICE.

Detailed Solution & Technical Explanation

Correct Answer: B

Global routing models the chip as a grid of bins (G-cells) to optimize wire topology and avoid congestion. Detailed routing follows the global routing guide to layout actual polygon metal paths, resolving design rule constraints (DRC) like spacing and width.

Q190: Non-Default Routing Rules (NDR) for Clock Nets

ADVANCED 60%

Why are **Non-Default Routing Rules (NDR: 2W2S - Double Width, Double Spacing)** applied to clock distribution nets during CTS and routing?

- A. To make clock wires look visually distinct in GUI.
- B. **Double Width (2W)** reduces wire resistance, minimizing clock insertion delay; **Double Spacing (2S)** dramatically cuts coupling capacitance (C_c) to adjacent tracks, eliminating crosstalk delay jitter and noise glitches.
- C. To double the clock operating frequency.
- D. To prevent scan chain shifting.

Detailed Solution & Technical Explanation

Correct Answer: B

Clock nets toggle every single cycle and are highly sensitive to skew and jitter. Doubling wire width reduces resistance (R), while doubling wire spacing reduces coupling capacitance to neighboring aggressors, isolating the clock from crosstalk.

Q191: Well Tap Cells vs Standard Filler Cells

ADVANCED 60%

What are the distinct physical functions of **Well Tap Cells** and **Standard Filler Cells**?

- A. Filler cells provide power; Tap cells provide clocks.
- B. **Well Tap Cells:** Connect V_{DD} to the N-well and V_{SS} to the P-substrate at regular maximum distance intervals to prevent CMOS **Latch-Up**; **Filler Cells:** Inserted into empty spaces in placement rows to ensure continuous N-well/P-well diffusion continuity and layer density for DRC compliance.
- C. Both are functional logic gates.
- D. Tap cells are only used for scan chains.

Detailed Solution & Technical Explanation

Correct Answer: B

Tapless standard cell libraries require separate Well Tap cells placed at maximum pitch intervals (e.g. Every $30\mu\text{m}$) to bias wells and prevent parasitic PNP SCR latch-up. Filler cells contain no transistors; they maintain layer density and well continuity in empty floorplan gaps.

Q192: Physical DRC vs LVS Signoff Verification

ADVANCED 60%

What is the difference between **DRC (Design Rule Checking)** and **LVS (Layout Versus Schematic)** signoff checks?

- A. DRC checks timing; LVS checks power.
- B. **DRC:** Verifies that the physical layout polygons satisfy all foundry geometric manufacturing rules (minimum metal widths, spacings, enclosure, density); **LVS:** Extracts the transistor-level circuit from physical layout polygons and proves 1-to-1 schematic/netlist connectivity and device sizing equivalence.
- C. LVS is run only on PCBs.
- D. DRC can be skipped if timing passes.

Detailed Solution & Technical Explanation

Correct Answer: B

DRC guarantees the layout can be manufactured without fabrication defects (shorts, opens, lithography errors). LVS extracts devices and nets from the physical layout and compares them against the post-

synthesis gate-level netlist to prove correct physical connectivity.

Q193: Parasitic Extraction and SPEF Format

ADVANCED 60%

What is **SPEF (Standard Parasitic Exchange Format)**, and how is it used in the design flow?

- A. A format for describing testbenches.
- B. An IEEE standard ASCII format that contains extracted parasitic wire resistances (R), grounded capacitances (C_g), and cross-coupling capacitances (C_c) for every routed net in the layout, used by PrimeTime to perform accurate signoff STA.
- C. A floorplan description file.
- D. An FPGA programming bitstream.

Detailed Solution & Technical Explanation

Correct Answer: B

After detailed routing, extraction tools (e.g. StarRC) calculate exact wire parasitics based on physical metal shapes and dielectric layers, formatting them as a `.spef` file for back-annotation into PrimeTime for final timing signoff.

Q194: Engineering Change Orders (ECO) Methodology

ADVANCED 60%

Late in the physical design cycle, a minor RTL logic bug is discovered after full routing: Why is an **ECO (Engineering Change Order)** flow preferred over restarting full synthesis and PnR?

- A. Re-running synthesis and PnR is illegal under foundry rules.
- B. An ECO modifies only a few gates using pre-placed **Spare Cells** or minimal local cell additions, preserving the fully closed timing, clock trees, and layout routing of 99.9
- C. ECOs run without an SDC file.
- D. ECOs eliminate DRC rules.

Detailed Solution & Technical Explanation

Correct Answer: B

Full re-implementation causes chaotic disruption to placed cells and clock trees ("timing closure roulette"). An ECO netlist patch modifies only the affected logic cone and routes minimal local wires, preserving all existing closed timing.

12 FPGA Architecture & ASIC vs FPGA

Q195: Look-Up Table (LUT) Combinational Architecture

EASY 10%

In modern FPGA architectures (e.g. AMD/Xilinx UltraScale+, Intel Stratix), what basic hardware structure implements arbitrary combinational logic functions?

- A. Standard cell NAND gates.
- B. **SRAM-based Look-Up Tables (LUTs)** (e.g. 6-input LUTs that can implement any arbitrary 6-input Boolean function as a 64×1 -bit memory with a 64-to-1 MUX).
- C. Hardwired microprocessors.
- D. Dynamic RAM capacitors.

Detailed Solution & Technical Explanation

Correct Answer: B

FPGAs do not have individual AND/OR gates. Combinational logic is implemented using SRAM-based

Look-Up Tables (LUTs). A 6-input LUT contains 64 SRAM bits initialized with the truth table of the Boolean function, addressed by the 6 inputs.

Q196: FPGA vs ASIC Commercial and Technical Trade-Offs

EASY 10%

What are the primary commercial and technical trade-offs of an **FPGA** compared to a custom **ASIC**?

- A. FPGA has higher unit cost at high volume, lower maximum frequency, and higher power dissipation, but zero Non-Recurring Engineering (NRE) mask costs and instant time-to-market with re-programmability.
- B. ASIC has zero upfront design cost.
- C. FPGA is faster and uses less silicon area than ASIC.
- D. FPGAs cannot implement sequential circuits.

Detailed Solution & Technical Explanation

Correct Answer: A

ASICs require millions of dollars in upfront mask/NRE costs but have low per-unit silicon cost, maximum performance, and minimal power at high volume. FPGAs have zero mask cost and are reconfigurable, but have higher per-unit cost and lower power/area efficiency.

Q197: Hard IP vs Soft IP in FPGAs

MODERATE 30%

Which of the following is an example of **Hard IP** in a modern FPGA?

- A. A synthesizable UART RTL module written in Verilog.
- B. Dedicated silicon blocks fabricated directly on the die, such as **DSP48 Slices**, **Block RAMs (BRAM / UltraRAM)**, **PCIe Hard Controllers**, and **Multi-Gigabit SerDes Transceivers**.
- C. A software C-program running on a host PC.
- D. An unconstrained clock net.

Detailed Solution & Technical Explanation

Correct Answer: B

Soft IP consists of synthesizable RTL code mapped into generic LUTs and flip-flops. Hard IP refers to custom, dedicated silicon blocks (BRAMs, DSP arithmetic units, high-speed transceivers) embedded in the FPGA fabric for optimal performance.

Q198: Dedicated Global Clock Buffers (BUFG)

MODERATE 30%

Why must all high-fanout clock signals in an FPGA design be routed through dedicated **Global Clock Buffers** (e.g. **BUFG**, **BUFGCE**)?

- A. General-purpose FPGA interconnect routing has high resistance and unpredictable segment delays, which would cause massive clock skew and hold violations; BUFG buffers drive dedicated, low-skew global clock spines.
- B. General routing fabric cannot carry electrical voltage.
- C. BUFGs convert digital clocks to analog waveforms.
- D. BUFGs eliminate the need for setup timing checks.

Detailed Solution & Technical Explanation

Correct Answer: A

General FPGA routing fabric consists of segmented pass-transistor routing switches that introduce large,

non-uniform delays. Dedicated clock buffers (BUFG) drive specialized, low-impedance clock distribution networks with minimal skew across the die.

Q199: Synchronous vs Asynchronous Reset Strategy in FPGAs

MODERATE 30%

Why do FPGA vendors (e.g. AMD/Xilinx) strongly recommend **Synchronous Resets** over Asynchronous Resets for FPGA design, unlike standard ASIC practice?

- A. Asynchronous resets do not work in FPGAs.
- B. FPGA logic slices (CLBs) have dedicated synchronous control set/reset pins; using synchronous resets allows the synthesis tool to pack control logic efficiently into dedicated flip-flop control sets and LUT inputs without wasting routing resources.
- C. Synchronous resets eliminate all setup checks.
- D. Asynchronous resets double dynamic power.

Detailed Solution & Technical Explanation

Correct Answer: B

FPGA flip-flops inside Configurable Logic Blocks (CLBs) share common control sets (clock enable, reset). Asynchronous resets prevent optimal register packing and restrict the synthesis tool from merging reset conditions into LUT logic.

Q200: Hazardous Nature of Inferred Latches in FPGAs

ADVANCED 60%

Why is an accidental inferred latch far more damaging in an **FPGA** implementation than in an ASIC?

- A. FPGAs do not possess physical latch cells in their standard slices; the synthesis tool must either emulate the latch using a slice flip-flop with feedback or construct a combinational loop across multiple LUTs, consuming excessive resources, creating severe timing hazards, and causing static timing analysis tools to fail.
- B. Latches cause the FPGA bitstream to erase itself.
- C. Latches reverse the polarity of the power supply.
- D. Latches disable the JTAG programming interface.

Detailed Solution & Technical Explanation

Correct Answer: A

FPGA slices are optimized for synchronous edge-triggered D flip-flops and LUTs. Inferred latches require either using flip-flops in non-standard latch modes (which disables surrounding registers in the slice) or creating combinational loops through LUT routing, which leads to race conditions and timing closure failures.